Combination of Workflow Modelling and Static Code Analysis to Assess the Effect of Changes in Industrial Automation Software for Recertification in the MedTech Domain

Bachelorarbeit
an der Fakultät für Maschinenwesen der Technischen Universität München.

Themenstellende/r	Prof. Dr.-Ing. Birgit Vogel-Heuser
	Lehrstuhl für Automatisierung und Informationssysteme
Betreuer/in	Eva-Maria Neumann, M.Sc.
	Lehrstuhl für Automatisierung und Informationssysteme
Eingereicht von	Michael Gnadlinger
	Freisinger Landstraße 90
	80939 München
	+4915252684025
Eingereicht am	14.10.2023 in München

Kurfassung

In der aPS-Domäne stellt die zunehmende Komplexität von Software eine kontinuierliche Herausforderung dar, die das Risiko für unvorhergesehene Querabhängigkeiten während der Umsetzung notwendiger Softwareänderungen erhöht. Dies bereitet insbesondere Unternehmen in der MedTech-Branche, die sich an die 'Good Manufacturing Practice'-Regulatorien innerhalb der Europäischen Union halten müssen, erhebliche Herausforderungen. Insbesondere, da diese Vorschriften verlangen, dass die Software einem formal dokumentierten Test- und Verifizierungsprozess, einer sogenannten Softwarequalifikation, unterzogen werden muss, um Konformität zu gewährleisten. Diese Formalität trägt jedoch zur zeitaufwändigen Natur etwaig notwendiger Rezertifizierungen von Softwareänderungen bei, welche in zeitkritischen Szenarien wie der Inbetriebnahme herausfordernd sein kann. Folglich ist das übergreifende Ziel dieser Arbeit die Unterstützung des Rezertifizierungs-Workflows durch die Integration von Methoden der statischen Codeanalyse in bestimmten Prozessschritten als Hilfsmittel für Ingenieure, um mögliche Auswirkungen von Änderungen zu bewerten und somit den Rezertifizierungsprozess zu verbessern. Das vorgestellte Konzept wurde anhand von zwei industriellen Fallstudien validiert und zusätzlich von Industrieexperten hinsichtlich seiner wahrgenommenen Nützlichkeit bewertet.

Abstract

In the aPS domain, the rising complexity of software poses a continual challenge, heightening the potential for unforeseen cross-dependencies during the implementation of necessary changes. This is particularly challenging for companies in the MedTech domain, which need to operate under the 'Good Manufacturing Practice' regulations within the European Union, as these regulations require that the software undergo a formally documented testing and verification process, known as software qualification, to ensure compliance. This formality contributes to the time-consuming nature of any necessary recertification procedures of software changes, which can be particularly challenging in time-sensitive phases such as commissioning. Consequently, the overarching objective of this thesis is to streamline the recertification workflow by integrating static code analysis methods in specific process steps as a tool for engineers to assess potential impacts of changes and thus enhance and assist the recertification process. The presented concept has been validated on two industrial-scale case studies and evaluated on the perceived helpfulness by industrial experts.

Table of Content

1. Motivation 1

2. Field of Investigation 2

2.1. Industrial Automation 2

2.2 Lifecycle, Evolution, and Changes of Control Software 2

2.3 Regulatory Constraints and Guidelines for the MedTech Sector (GMP, GAMP 5) 3

2.3.1. Regulatory Constraints: Good Manufacturing Practice (GMP) 4

2.3.2. Guidelines: Good Automated Manufacturing Practice 5 (GAMP 5) 5

3. Requirements for the Concept 7

4. State-of-the-Art and Related Work 10

4.1. Methods for Static Code Analysis of aPS Software 10

4.2. Analysis of Changes in Software and Their Impact 11

4.3. Workflow Models 12

4.3.1. Workflow: Definitions and Importance of Communication 12

4.3.2. Workflow-Modelling: Introduction to Business Process Model and Notation (BPMN) 13

4.3.3. Other Common Workflow Models 15

4.4 Research Gap 16

5. Concept of Implementing Static Code Analysis in the Workflow to Assess the Effect of Changes . 17

5.1. Generalized GAMP-Workflow in BPMN++ 17

5.1.1 General Approach to Change Management 17

5.1.2. Specifics on Project Change Management 18

5.1.3. Specifics on Operational Change Management 21

5.2. Integrating Static Code Analysis Methods in Critical Steps of a GAMP-Derived Workflow to Determine Change Impact 23

6. Introduction to Prototypical Implementation of Static Code Analysis Methods 25

7. Evaluation of the Concept of Implement Static Code Analysis in the Recertification Workflow 28

7.1. Introduction of Industry Partner 28

7.1.1 Description of Plant and Affiliated Software of Case Study 29
7.2. Certification Workflow of the Industry Partner and Comparison with GAMP 29
7.2.1. Description of Industry Partners' Workflow 30
7.2.2. Correlation With GAMP-Introduced Workflow 34
7.3. Analysis of Changes in Industry Partners' Software 35
7.3.1. Effective Changes Concluded from Empirical Analysis of Two Sub-Plants 35
7.3.2 Possible Changes Throughout the Life Cycle Given by an Expert 37
7.4. Industry Expert Interviews on the Helpfulness of the Concept 38
7.4.1. Introduction of Interview Partners for the Evaluation 38
7.4.2. Procedure of the Interview and Description of Use-Cases 39
7.4.3. Results of the Expert Interviews 41
7.5. Results of Evaluation and Discussion 43
8. Conclusion and Outlook 46
8.1. Conclusion of Research 46
8.2. Outlook for Further Research Projects 46
References 47
Appendix A. Abstraction of Industry Partners' Workflow in a Non-standard Decision Tree 53
Appendix B. Abstraction of GAMP-suggested Workflow in a Non-standard Decision Tree 54
Appendix C. Summary of Questionnaire used to Determine Background of Experts in Interviews 55
Eidesstattliche Erklärung 56

1. Motivation

automated Production Systems (aPS) are highly complex mechatronic machinery requiring close cooperation of multiple disciplines throughout the products' life cycle, further complicating their development and maintenance [1]. Such systems designed explicitly for the MedTech domain additionally underly strict regulations, within the European Union namely Good Manufacturing Practice (GMP), to ensure product quality and patient safety by calling for certain constraints like risk-management approaches, or rigorous validation and documentation processes [2]. Since changes to an aPS are inevitable, e.g. due to updated requirements, because of the long life cycle of these systems of several decades [3, p. 88], they become an integral part of the maintenance and effort. As the agile nature of software development and time pressure when unforeseen changes arise coincide, these changes are mainly performed in software rather than by a mechanical or electronic/electric solution [1]. This in turn results in the software needing to undergo the constraints laid upon the system by the regulations, especially validation and certification processes, multiple times - so-called recertification.

At the time recertification is performed by industry experts with a deep understanding of the overall plant functionality and software at hand, as they also need to consider possible propagations of a specific change through the software architecture to assess the extent of the required recertification. The ever-growing complexity of aPS [4] only aggravates this challenge and calls for a tool-assisted concept to aid with the decision-making process, while established automated testing processes from high-level language software development are not compatible with the specifics of the aPS domain.

The contribution of this thesis hereby is to develop a novel concept that aids communication and the general workflow surrounding required changes to software, which must adhere to certain regulatory standards. This will be achieved by introducing established practices for software analysis. Although both workflow modelling, as well as software analysis, are already well researched topics, there is currently a gap in the literature regarding the integration of these two areas. This thesis seeks to address this research gap and establish a meaningful connection between workflow modelling and software analysis.

The final evaluation will be on a German special purpose machine manufacturer from the field of medical engineering, centring its attention on conducting interviews and workshops with domain experts.

2. Field of Investigation

To gain a better understanding of the challenges and limitations in this domain, this chapter will provide an explanation of the theoretical framework relevant to this thesis, ranging from an overview of industrial automation and its software, the regulatory constraints of relevance, to workflow and workflow modelling.

2.1. Industrial Automation

The goal of Industrial Automation is to automate technical processes, a task achieved through the use of aPS, which encompasses the field of machinery and manufacturing plants. This thesis is specifically focused on designed-to-order production systems, which entail unique machinery that face distinctive struggles and challenges [5]. Because of their real-time capability and the tough industry environment, these systems are usually controlled using so-called “programmable logic controllers” (PLC). Within the IEC 61131-3 standard, three programming languages are defined for these PLCs: two textual languages (Instruction Language (IL) and Structured Text (ST)) as well as three graphical languages (Ladder Diagram (LD), Function Block Diagram (FBD) and Sequential Function Chart (SCL)) [6]. In the industry, various dialects may arise, such as Siemens' Structured Text Language (STL), but they still adhere to the IEC 61131-3 standard [7]. PLC software architecture is structured in Program Organization Units (POU) categorized by the IEC 61131-3 into Program (PRG), Function Block (FB), and Function (FC), where the PRG acts as a starting point to call subsequent POUs and the differentiating factor between FBs and FCs being, that FBs can internally store values over several cycles. Additionally, variables can be declared either within existing POUs or made globally accessible through the use of data blocks (DB)

The inherent challenge with aPS lies in their interdisciplinarity, as they require the integration of mechanical, electronic/electric, and software engineering into a single project [8]. This interdisciplinary development process necessitates extensive information exchange among these different engineering fields, each with its unique tools and expertise, making such engineering projects vulnerable to delays or cost overruns should this information exchange be inefficient or even fail [9].

2.2 Lifecycle, Evolution, and Changes of Control Software

For aPS, the definition of its useful lifetime has been adopted from the cyclical consideration of the product development context, where a key aspect of a life cycle is its subdivision into discrete phases [10]. For the aPS-domain, these phases are typically coined as planning/construction,

commissioning/start-up, and operational phase and thereby span from the conceptualization of a project until its decommissioning.

Because this life-cycle of manufacturing plants can span over several decades [3, p. 88] aging due to physical effects on mechanical and electrical hardware is a true concern and integral part of hardware maintenance efforts [11]. These maintenance efforts usually result in changes due to the fact that replacement parts are often not identical and therefore not interchangeable without issues [1]. These required changes are commonly performed in software because they can be done during run time or even remotely [1]. Challenges additionally arise through the different aging processes and, thereby different life-cycles and changing frequencies of the three disciplines (software, mechanical, and electric engineering) and the fact that alterations in one discipline can affect other fields (Figure 1) [12].

Vogel-Heuser et al. identified several other key drivers for changes throughout its long life-cycle such as shifting market requirements or even changing legislator regulations and defined “evolution” as the adaptation of an aPS due to the need for change [1]. Furthermore, Vogel-Heuser et al. incorporated the taxonomy by Buckley et al. [13] to discretely differentiate between changes that can be foreseen e.g. during development, and changes that arise unanticipated commonly during commissioning. Especially these unanticipated changes are of great interest as the combination of time pressure and breakdown of workflow call for assistive tools to prevent any oversights in these critical stages.

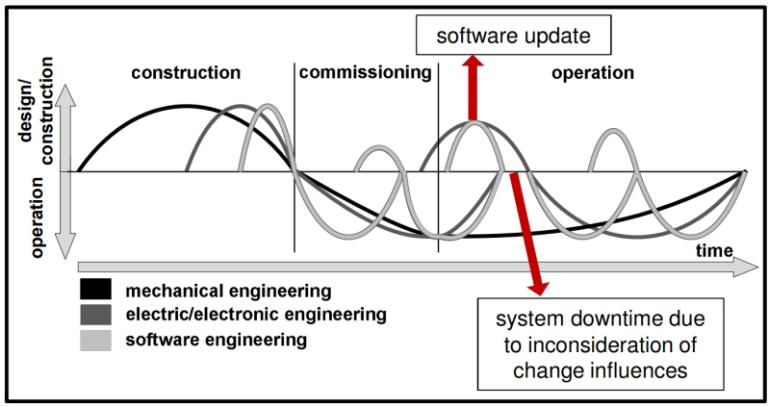

Figure 1: Life cycle differences of interdisciplinary elements of aPS [12]

2.3 Regulatory Constraints and Guidelines for the MedTech Sector (GMP, GAMP 5)

In this chapter, the European Union's regulatory framework GMP, concerning quality assurance of medicinal products, is being introduced and discussed with its impact regarding software development in general and, more specifically, its intricate consequences it gives rise to in the case of software

changes. Afterwards the GAMP 5 guideline is being presented with the methods suggested within it to help companies to be compliant with GMP and similar regulations.

2.3.1. Regulatory Constraints: Good Manufacturing Practice (GMP)

The predominant constraints for manufacturers of aPS in the MedTech domain, like the industry partner for this thesis (see Chapter 7.1.), are the regulations found within the frameworks and guidelines of the Good Manufacturing Practice (GMP) of the European Union (EU). This regulation aims to ensure that medicinal products are fit for their intended use and do not place patients at risk due to inadequate quality, by correctly incorporating good manufacturing practices and quality risk management [14]. Furthermore, for computerized systems which replace manual operation the GMP regulations require there to be no resultant decrease in product quality, process control or quality assurance, as well as no increase in the overall risk of the process [15]. The basic requirements of GMP, of most importance for this thesis, are a) that all manufacturing processes are systematically reviewed and shown to be capable of consistently manufacturing medicinal products of the required quality and comply with their specifications and b) that significant changes to the process are being validated [16]. This quality requirement of e.g. aPS can be assured by complying with the principles of good manufacturing practice [2]. Compliance with GMP underlies the European Medicines Agency (EMA) and requires qualification and validation documentation which should be approached using quality risk management [17].

Quality risk management hereby is defined as a systematic process for assessment, control, communication, and review of risks to the quality of the medicinal product, which can be applied both proactively as well as retroactively [4]. This methodology of quality risk management is also to be used to evaluate planned changes and to determine the potential impact on product quality, documentation, validation and on other systems to avoid unintended consequences and to plan for any necessary process validation or requalification [6]. Therefore, for any changes in the process steps should be taken to demonstrate its suitability for routine processing to yield a product consistently of required quality [5].

For all stakeholders within the production of medicinal products it is also of importance, that GMP applies to all life cycle stages of the product, laid out from the first investigational manufacturing through to the products discontinuation [4]. This mainly affects suppliers of aPS for medicinal products subject to GMP regulations when it comes to maintenance of their production systems which in turn are also subject to GMP for their complete life cycle of the production. This subsequently also applies to the above-mentioned qualification and validation activities, including risk assessments, which should be planned to take the whole life cycle into consideration [3], [6].

In regards to suppliers of aPS, the significantly different life cycles of long-living hardware and rapidly updated software during the operation of aPS (Figure 1)[1], can lead to increased workload due to the

consistent demand of GMP for evaluating these rapidly occurring software changes in the process of medicinal products throughout the long lifetime of the production plant.

2.3.2. Guidelines: Good Automated Manufacturing Practice 5 (GAMP 5)

GAMP 5 encompasses a set of Guidelines by the International Society for Pharmaceutical Engineering (ISPE) that introduce methods and procedures to help with compliance with regulations for the pharmaceutical sector. Especially the issue "GAMP 5 a risk-based approach to compliant GxP computerized systems" is of relevance for this thesis as it additionally specifically addresses software aspects like a whole appendix dedicated to the development phase [18, Appendix D], as well as solutions for change and configuration management [18, Appendix M8].

The abbreviation "GxP" herein envelops several different regulatory domains, such as, but not limited to, GMP (Good Manufacturing Practice), GCP (Good Clinical Practice), GLP (Good Laboratory Practice), for diverse regulatory jurisdictions like the FDA (Food and Drug Administration) in the USA or EU Directives [18, p. 22 f.]. Therefore, these guidelines are widely applicable in the MedTech domain and have validity beyond the specific use-cases discussed in the evaluations in Chapter 7.4.2.

The two predominant methods introduced are a) the "risk-based approach", in accordance with the quality risk management in the EU regulations [17], and b) "life-cycle approach". Risk-based approach follows the mentality, that measure should be appropriate to the risk introduced to patient safety, product quality and data integrity [18, p. 20 f., p. 43, p. 247 f.]. The life-cycle approach concentrates on considering the long life-cycle of plants in the decision-making process, to guarantee compliance of the machinery throughout its lifetime [18, p. 25 ff.]. The Guideline provides differentiation of the uniqueness of the supplied machinery and explicitly emphasises the involvement of the plant manufacturer for custom applications (GAMP Category 5 of 5 categories) (Figure 2) [18, p. 65]. This requires proper, efficient communication between both plant user and plant supplier teams, as was the observation in the book on general workflows authored by White and Fischer in [19].

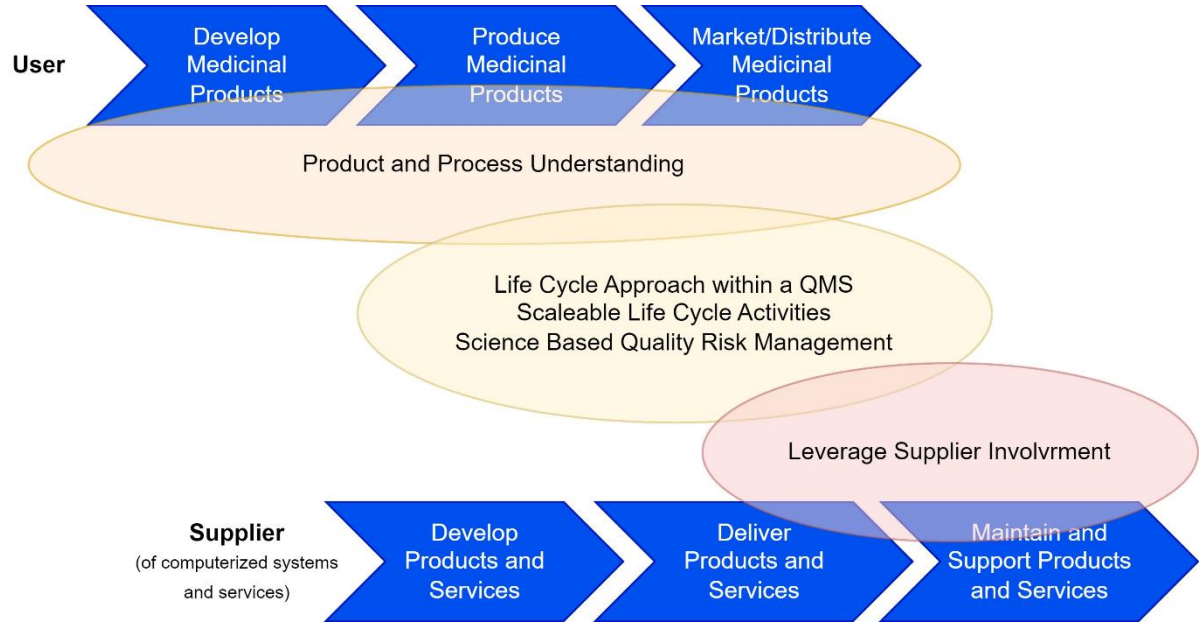

Figure 2: Involvement of plant supplier throughout the life-cycle of the machinery; adapted from [18, p. 19]

Both approaches are of particular importance when it comes to managing changes, where all changes, whether to software or hardware, are suggested to be subject to a formal change control process. With the level of rigor in this process being based on the nature, risk and complexity of the specific change at hand [18, p. 43]. As software complexity naturally grows throughout the various stages of development, the level of complexity in software changes is, in turn, influenced by the project phase, prompting the need for varied approaches.

3. Requirements for the Concept

For a concept to be considered as helpful and thereby accepted into an established workflow by its user, it must fulfil certain requirements. The following eight requirements have been derived from challenges in the industry as well as based on the constraints introduced in the Field of Investigation (see Chapter 2.).

Requirement 1: The concept should be applicable regardless of the stakeholders involved.

A key requirement for all concepts concerned with aPS is the inherent interdisciplinary nature of these systems, with a close relation of software engineering, electrical and electronic engineering and general mechanical engineering throughout their whole life cycle [1]. Additionally, interactions with individuals who may not have as strong of a technical background, such as management and customers, can take place at various stages of the plant life cycle. These instances might arise during development, when changes needed to be escalated to higher levels, or after commissioning and hand-over to the customer. Winkler et al. concluded, from observations of an industry partner, that these different fields are heterogenic by using individual tools and data models, which hinders collaboration. As such linking these diverse disciplines, of different backgrounds, in overlapping, collaborative scenarios plays a pivotal role in assuring overall quality and efficiency [20].

Requirement 2: Scalability of the concept to cover large, industrial-scale, projects.

The emergence of globalization and therein the rise in competition in the plant manufacturing industry over the last decades has shifted the market to a customer's market which called for increasingly more complex and feature rich production systems [4]. Thus, concepts for supporting automation engineers need to be scalable to these sizes [1], while still remaining quickly understandable and providing a suitable overview.

Requirement 3: The concept should be able to be applied/created automatically.

As the concept aims to support personnel at specific tasks as well as enhance communication between stakeholders to overall streamline the software change process, these gains in efficiency must not be lost by introducing additional workload into the workflow. By automatically creating most, if not all, aspects of the concept, a net efficiency gain to the workflow can be expected. Additionally, the scalability requirement of the aPS domain may warrant automation because of the sheer size of the systems in question. Given its time-criticality, special considerations are to be placed on the commissioning phase.

This phase usually requires time intensive adaptations and fixes to the behaviour of mechanical or electrical sub-systems, which can be solved faster through agile software changes [21]. Consequently, the application of code analysis at various workflow steps in the commissioning phase that adds workload, will not pose any beneficial advantage to the end user.

Requirement 4: Independency from IEC 61131-3 programming language.

The IEC 61131-3 standard covers five different languages (see Chapter 2.1.), both graphical and textual, for use on the Programmable Logic Controller (PLC). As all five of these languages are utilized by aPS manufacturers, the efficacy of the static code analysis approach needs to be universally applicable, regardless of the specific programming language in use.

Requirement 5: Concept should be applicable throughout the whole project life cycle.

aPS are designed to operate for several decades, thus their evolution is often referred to as life-cycle with distinct phases [22]. (see Chapter 2.2.) Throughout all these phases changes to the system can occur, each presenting distinct difficulties and challenges [1]. One phase of particular interest in the commissioning phase, where often unexpected changes arise, that need to be addressed under time pressure [23]. But also, the differentiation between pre- and post-handover of the plant to the customer is a key requirement for the concept, to make sure it can cover all possible change scenarios.

Requirement 6: The concept should be supportive in specific work steps as well as in communication.

Three-quarters of white collar time is spent on communication [19]. This fact shows, that communication is an integral part of all workflow processes and is only amplified in breakdown scenarios of basic workflows [19]. Within the context of aPS, breakdowns predominantly manifest during the initial start-up phase, as this phase is dominated by the emergence of unanticipated design flaws that demand immediate on-site resolutions, leading to unplanned software changes [1]. (see Chapter 2.2.)

Requirement 7: The concept should be reasonably useable for documentation purposes.

Documentation is an integral part of the regulatory framework, Good Manufacturing Practice (GMP), for medical devices, to e.g. capture changes to critical components and always assure a user-safe quality of the end product. These documents additionally serve as detailed provisions on inspections. [14] (see Chapter 2.3.)

In the context of changes to the manufacturing system, including any software, the European Medicine Agency (EMA) stipulate a risk based approach, on which the taken measures should be proportional to

the expected risk level on the quality of the end product [17]. The reasoning of the required risk assessment is to be documented, which in turn means that a beneficial part of any concept concerned with changes upon a system underlying regulation, would be a reasonably well usability as documentation.

Requirement 8: The primary information should be presented in an understandable way.

Due to the interdisciplinary nature of aPS and its associated challenges [8] (see Chapter 2.1.), the concept should not impede information exchange, but rather be easily comprehensible even by non-experts. Additionally this requirement further widens the applicability of the concept onto customers, which is another important communication pipeline as laid out by the GAMP 5 Guidelines [18].

Quickly and easily understandable models are also an essential requirement for them to be helpful in time-critical, stressful situations like the start-up phase. Especially during this phase, unplanned changes often occur, making it all the more critical for the concept to be helpful without excessive time investment. (see Chapter 2.2.)

4. State-of-the-Art and Related Work

By using the field of investigation established in the preceding Chapter 2 as a basis, existing research is assessed and evaluated to determine its relevance to the domain and the problem at hand. This is split into three sub-sections: firstly, code analysis methods, followed by the analysis of changes in software and their impact and lastly an introduction of workflow and workflow models.

4.1. Methods for Static Code Analysis of aPS Software

Static code analysis is an examination process of software properties without executing the program and have existed in computer science since the 1980s [24]. Adaptations to the aPS domain have been proven to be challenging due to the unique requirements of aPS comprehensively analysed through large-scale questionaries and case-studies by Vogel-Heuser et al. [8, 25, 26]. Thus approaches usable for high-level programming languages needed to be adapted and enlarged to be suitable for analysis of aPS software [27]. Although not strictly separated, the resulting analysis methods can be grouped into metric-based and structural analysis. Metric-based analysis herby focuses on quantifying software quality attributes (e.g. reusability) through measuring certain software properties (e.g. number of comments) [28]. Common software quality attributes are defined in within ISO 25010 [29]. Different software quality metrics try to approach describing certain quality attributes through different constellations of measured software properties, but in the end output a numerical value to the user as an analysis result [30–32]. Although these numerical values can be represented using diagrams and are proven to assist in software development [26], they still require an in-depth knowledge in code analysis as well as software development to correctly interpret the presented values, as target conflicts might arise, giving the user contradicting results and requiring additional structural analysis [28]. In turn violating *Requirement 1* and if not comprehensively processed into understandable diagrams *Requirement 8*. On the other hand, structural analysis methods mainly focus on the software architecture and the systems' non-functional properties [33]. Their representability in the form of graphs, consisting of nodes for the POUs and edges for the dependencies between those POUs, makes structural code analysis inherently easier understandable without a background in software development (*Requirement 1*). Additionally, specific design patterns for PLC software (e.g. 'central unit', 'cornflower') embedded in these graphs have been derived by Fuchs et al. [34] and further formalized [35] to assess software quality, as used in the analysis part of this thesis in Chapter 7.3. For the concept of this thesis three graphical representations of structural analysis have been chosen to encompass all necessary information: (i) direct data exchange (DDE) in the form of *Call Graphs*, (ii) indirect data exchange (IDE) in the form of *IDE Graphs*, and (iii) evolutionary software version comparison in the form of *Difference Graphs*. In Chapter 5.2. the specifics of each graph is presented and discussed on their fulfilment of the Requirements.

4.2. Analysis of Changes in Software and Their Impact

The analysis of how changes propagate through software is a vast and still expanding field. One such approach, which in further work has been enhanced to be applicable on aPS, is by the Karlsruhe Institute of Technology (KIT). Especially because of its applicability on aPS it is of special interest for this thesis and will the introduced to then review it regarding the requirements derived in Chapter 3.

Karlsruhe Architectural Maintainability Prediction - KAMP

Karlsruhe Architectural Maintainability Prediction (KAMP) was first introduced in a cooperation between the Forschungszentrum Informatik (FZI), and the KIT is an approach to evaluate maintainability based on architectural analysis of the software at hand [36, 37]. It specifically focuses on estimating the costs of a given change request to an already established software structure, with the goal to automatically and independently of any expertise estimate the change effort [36]. As promising as the evaluation of KAMP by Rostami et al., regarding both its precision and completeness was [38], this approach presents two major shortcomings for the requirements of the concept needed in this paper: (i) KAMP is designed for the high-level language software domain and therefore is not per-se applicable to the field of investigation, particularly the domain of aPS, in this thesis and its implicit challenges (see Chapter 2.1) [37, 39]. (ii) KAMP is designed for the target group of software architects and the project managing level, who have vastly different requirements than the target group of software developers and commissioning engineers for aPS in this thesis [36] [40, p. 22]. E.g. the critical commissioning phase was no point of concern. Hence KAMP is generally not applicable to the field of investigation as derived in Chapter 2.

Karlsruhe Architectural Maintainability Prediction for automated Production Systems – KAMP4aPS

These shortcomings warranted the introduction of KAMP – "for automated Production Systems" (KAMP4aPS) as an extension to KAMP with the aim of being applicable for scenario-based architecture analysis in the aPS domain [40]. With the combination of metamodels, describing the plant at hand, and change propagation rules for the specific metamodel, a comprehensive task list with all potentially impacted elements (software, mechanical, as well as electronic/electric) is being created [41, p. 4]. Koch introduced two metamodels of different levels of abstraction and determined the more specific metamodel, with precision and recall of over 90%, to be significantly more accurate [41, p. 20]. The drawback being, that this metamodel is specifically designed to represent the open demonstrator plant "extended Pick & Place Unit" (xPPU), which, although designed explicitly for case studies of plant and machine automation [42], does not guarantee compatibility of the metamodel with real-world production plants, especially highly individualised special purpose machines as present in the MedTech domain.

This lack of industrial evaluation and Kochs' outlook on further research on a universally applicable metamodel [40, p. 71, p. 87] makes KAMP4aPS not yet viable for industrial use and thus contradicts *Requirement 2* as introduced in Chapter 3. The only way to currently guarantee precision on an industrial scale is to further specify the metamodel [41, p. 41], thereby making it even less broadly applicable, resulting in the need for plant manufacturers to design their own fitting metamodels for each new plant, which in turn require their own change propagation rules [41, p. 42]. This not only adds a high workload level, contradicting *Requirement 3* for this thesis' concept of not adding additional workload, but also requires specialised employees as the change propagation rules are implemented using Java [40, p. 88].

Other analysis methods can be divided into four categories by increasing granularity: task-based project planning approaches, architecture-based project planning, architecture-based software evolution, and scenario-based architecture analysis [11]. Task-based project panning approaches are generally found to be too coarse-grained for reliable and accurate impact analysis [11]. Where architecture-based project planning and software evolution approaches, such as those proposed in [43] and [44], lack sufficient automated generation of found results, limiting their scalability as outlined in *Requirement 3*. While scenario-based architecture analysis approaches, as presented in [45] or ALMA introduced in [46] alone, do not include a formal description of the software architecture needed to determine the propagation of changes throughout the software, making it hard to meet *Requirement 7* and *8*, as a formal description would additionally need to be developed and validated.

4.3. Workflow Models

A sequence of processes can be described within a so-called workflow and hence be modelled using various modelling languages and approaches. After capturing the essence of workflows, BPMN, one such modelling language, is being introduced and its advantages outlined to reason why specifically it has been selected for this thesis in favour of other commonly used modelling languages.

4.3.1. Workflow: Definitions and Importance of Communication

The concept of workflow, in its essence, is defined as a specific sequence of processes through which work traverses. White and Fischer introduce two ways such a workflow can be perceived: "ready-to-hand" or "present-to-hand". Where the former describes the utilization of a tool without the user being aware of the process steps, while the latter signifies the moment when the workflow becomes explicitly conscious. Workflow only becomes apparent to the user, if a breakdown occurs which is usually being resolved through communication [19]. In the scope of aPS, breakdowns typically happen during the

start-up phase, wherein unforeseen design bugs necessitate prompt on-site resolutions. Given this inherent time constraint, bugs of any nature in these interdisciplinary systems, encompassing mechanical, electrical and software engineering, are primarily addressed with changes on the software side, as ad-hoc adaptations of hardware are often not feasible on such short notice. The specific time frame of start-up and commissioning in the long lifecycle is especially interesting, since they significantly compromise the outcome of the project and often is underestimated [47] (*Requirement 5*). Additionally, but not as crucial for the focus of this thesis, foreseeable changes over the long operational lifecycle of production systems, of more than 30 years, are unavoidable [1].

This combination of "present-to-hand" workflow and extensive reliance on cross-disciplinarity collaboration to address bugs, warrants a significant investment in communication among all relevant stakeholders. These stakeholders may include teams within the company, the customer corporation, or outsourced teams (*Requirement 1*). Vogel-Heuser et al. demonstrate that a lack in efficient information exchange and inadequate cooperation can lead to delays, cost overruns und quality issues [9]. In fact, White and Fischer go to the extent of asserting that "if people don't communicate work doesn't flow" [19] (*Requirement 6*). The objective of enhancing interdisciplinary communication encounters a significant obstacle in the form of expertise mismatch in tools used across different disciplines. This discrepancy becomes evident through inherent compressibility issues experienced by individuals from outside those respective fields and in turn hampers efficient cooperation. [20, p. 48]

4.3.2. Workflow-Modelling: Introduction to Business Process Model and Notation (BPMN)

BPMN is a flowchart-like modelling language developed by Business Process Management Initiative (BPMI) which since 2005 merged to the Object Management Group (OMG) [49]. BPMN and its subsequent successor BPMN 2.0 basically comprise of Start-, Intermediate- and End-Events which are connected through certain activity-boxes and gateways if divergences are required. Additionally, a big advantage is present in the form of "swimlanes", overarching boxes that group parts of the workflow into its affiliated business in Business-to-Business (B2B) workflow scenarios [50, p. 28 ff.]. With this simple yet comprehensive set of elements BPMN has been found to be easily comprehensible even by none domain experts, yet formal and semantically strong enough to convey all necessary information of a process [51] (*Requirement 1*, *Requirement 8*).

BPMN+I

Vogel-Heuser et. al. introduced BPMN^{+I} (+I for innovation), as an enriched model of the BPMN 2.0 notation, with the key intention to obtain a more objective assessment of a given workflow and increase

transparency in decision-making. This need arose from the finding of Mathieu et.al., that aPS-domain, partly due to its interdisciplinarity, functions on a complex network of teams, so-called Multi-Team Systems (MTS). This reliance on interpersonal processes and relationships within a MTS, is being split by BPMN^{+I} into two categories: a) organizational aspects and b) psychological aspects. These categories can then be separately assessed and rated using a table, enhancing the depth of understanding and transparency. [52]

BPMN++

In 2021, Vogel-Heuser et. al. further revised this workflow model into the BPMN++ standard, with the addition of new notational elements, sub-diagrams as well as being able to integrate the formally separate assessment table directly into the graphical notation, to quicken the identification of relevant information [53, p. 2]. Of specific interest to this thesis is the new partition of the workflow into the two separate sub-diagrams denoted as the "cooperation diagram" and the "network of actors", with focus on the enhancements over previous BPMN versions, to communicational notations (*Requirement 6*). Wherein the "corporation diagram" includes the common workflow notation of events, activities and gateways, enhanced with additional elements to assign specific actors/teams to activities and assess technological and psychological aspects like necessary inter-team communication between these actors/teams which is depicted by a link with a double-sided arrow (Figure 3) [53, p. 8 f, Table 1, Table 3]. The "network of actors" on the other hand solely focusses on the inter-organisational relation of the companies involved in the cooperation process and lists all their actors/teams grouped into their associated companies and locations (Figure 4) [53, p. 9 f, Table 3]. To reduce unnecessary additional work, Vogel-Heuser et al. gives the outlook, that the "network of actors"-diagram could be derived automatically [53, p. 26].

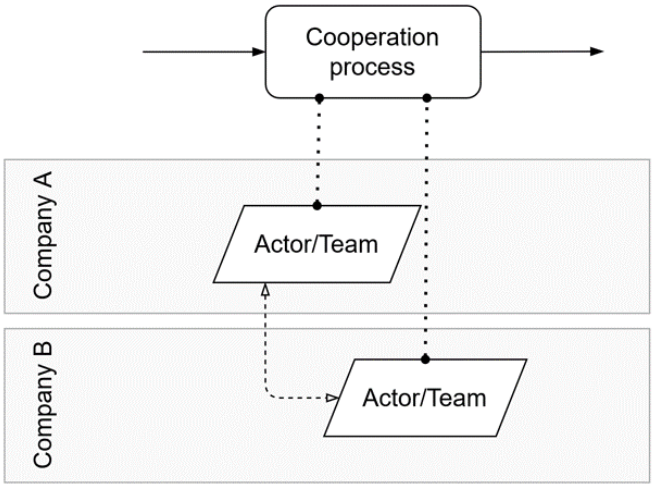

Figure 3: Depiction of the BPMN++ model of a cooperation diagram with two involved actors/teams from different companies needing to communicate to work at the same process step; adapted from [53, p. 8]*; enriched by communication-arrow*

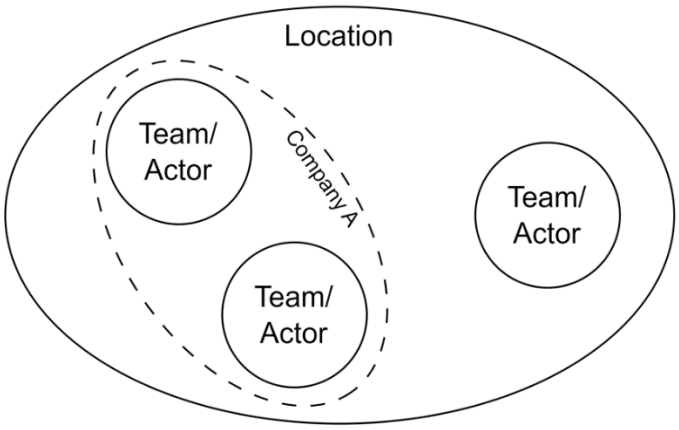

Figure 4: Depiction of a BPMN++ model of a "network of actors" with two involved companies and three teams needing to cooperate at the same location; adapted from [53, p. 10]

4.3.3. Other Common Workflow Models

The research field of Model-Based System Engineering (MBSE) covers various other standardised modelling languages to formalise the decision-making process throughout the life cycle. While the findings of Fischer show inherent differences in the methodical development approach by the different disciplines commonly involved in aPS, frequently adopted workflow models designed for interdisciplinary projects include V-Model XT and PI-steps [54].

V-Model XT

The V-Model XT excels in controlling cost and quality throughout the life cycle of a product due to its iterative design with decomposition steps of the system into discrete phases on the downwards branch of the “V” and associating integration and test steps on the opposing upwards branch. While it falls short in explicitly modelling responsible stakeholders for each step and any necessary communication between them, contradicting *Requirement 6* [55, p. 5].

PI-Steps

PI-Steps, on the other hand, present a modelling language of greater detail and finer granularity, including stakeholders for specific steps. It especially stands out for the comprehensive coverage of the life cycle until the very end, the decommissioning phase, and supports the modelling of tools for the workflow along the life cycle. However, it lacks a formal way of adding documentation to workflow steps, which is a crucial part of the whole workflow in regulated domains, as outlined in the Field of

Investigation in Chapter 2, and, therefore, not viable to be used in the environment of this thesis [56, p. 36].

Decision Trees

Another form of modelling a workflow is the so-called Decision Trees. While they are the simplest model to understand among those presented and can be enhanced by adding custom elements to represent stakeholders and communication interdependencies, this leads to a non-standardized model, making it hard to meet *Requirement 7* and *8*, without introducing a new, standardised formalisation. For the scope of this thesis, it was found to be sufficient to use this informal model (Appendix A and Appendix B) for evaluation purposes, but it cannot be guaranteed to be generalised to other use cases.

4.4 Research Gap

From the comprehensive overview of the state-of-the-art and related work in the previous chapters it can be concluded that there is no scientific approach to implementing static code analysis methods into regulated MedTech workflows. Mainly because the research focuses the general applicability of the aPS domain and mostly disregard any further regulatory constraints. Furthermore, most parts of existing approaches do not satisfy the Requirements derived in Chapter 3, thus needing a careful selection of solutions from the research fields to successfully combine into the concept presented in the following Chapter 5. This concept of presenting suitable static code analysis methods for specifically identified processes in a GxP regulated recertification workflow aims to fill this identified gap in research.

5. Concept of Implementing Static Code Analysis in the Workflow to Assess the Effect of Changes

In this chapter the concept of combining workflow modelling and static code analysis is described by firstly introducing the GAMP-suggested workflows for any company that underlies GxP regulatory constraints (Chapter 5.1.). These workflows are then being enhanced by introducing static code analysis methods and outlining and justifying which explicit processes in these workflows they can be assisted (Chapter 5.2.).

5.1. Generalized GAMP-Workflow in BPMN++

The GAMP 5 guidelines never explicitly draw up a workflow, rather the suggested procedure to handle changes in software are circumscribed from various aspects throughout their associated appendices. Most notably, firstly a general approach of managing changes is offered (Chapter 5.1.1), which in a second step is then further elaborated by differentiating between so-called “project change management” (Chapter 5.1.2.) and “operational change management” (Chapter 5.1.3.). This chapter tries to capture this structure and present the translation and formalisation of the suggested workflows into BPMN++.

5.1.1 General Approach to Change Management

In order to render the implementation of GMP regulations both practicable and conducive to competitiveness, while avoiding undue burdens, the approach follows the mentality of progressively enhancing the formality and rigor of change control throughout the various developmental phases of the project [18, p. 150 f.]. The ISPE posits the concept of a handover point as the ultimate stage for potential differentiation, thereby facilitating the distinction between changes occurring during the projects’ developmental phase and those emerging once the aPS is already operational. Consequently, the latter is denoted as “operational change management”. An exact definition for this handover point is never explicitly mentioned, but it can be reasonably derived that it is comparable to the point in time, when the plant manufacturer finishes the final qualification and therefore approval to be fit for operation. The development phase is subsequently finer divided in a so-called “approved prototyping” and a “documented design proposal” stage. The contention here lies in the notion that prototyping work, as opposed to formal development, primarily embodies an iterative and evolutionary process that should remain unburdened by the introduction of non-productive tasks [18, p. 151]. Consequently, it is deemed exempt from any stringent formality. Contrary, changes that take place outside of the prototyping phase during development should fall under the purview of what is termed "project change management." Notably, it is recommended in this context to refrain from employing formal change control and, instead,

maintain all documents in a draft state, allowing for agile adaptations as necessary. The overarching principle for GAMP 5 prevails, as the formality within these change management approaches span on a spectrum ranging from informal team meetings, all the way to formally written and documented change requests.

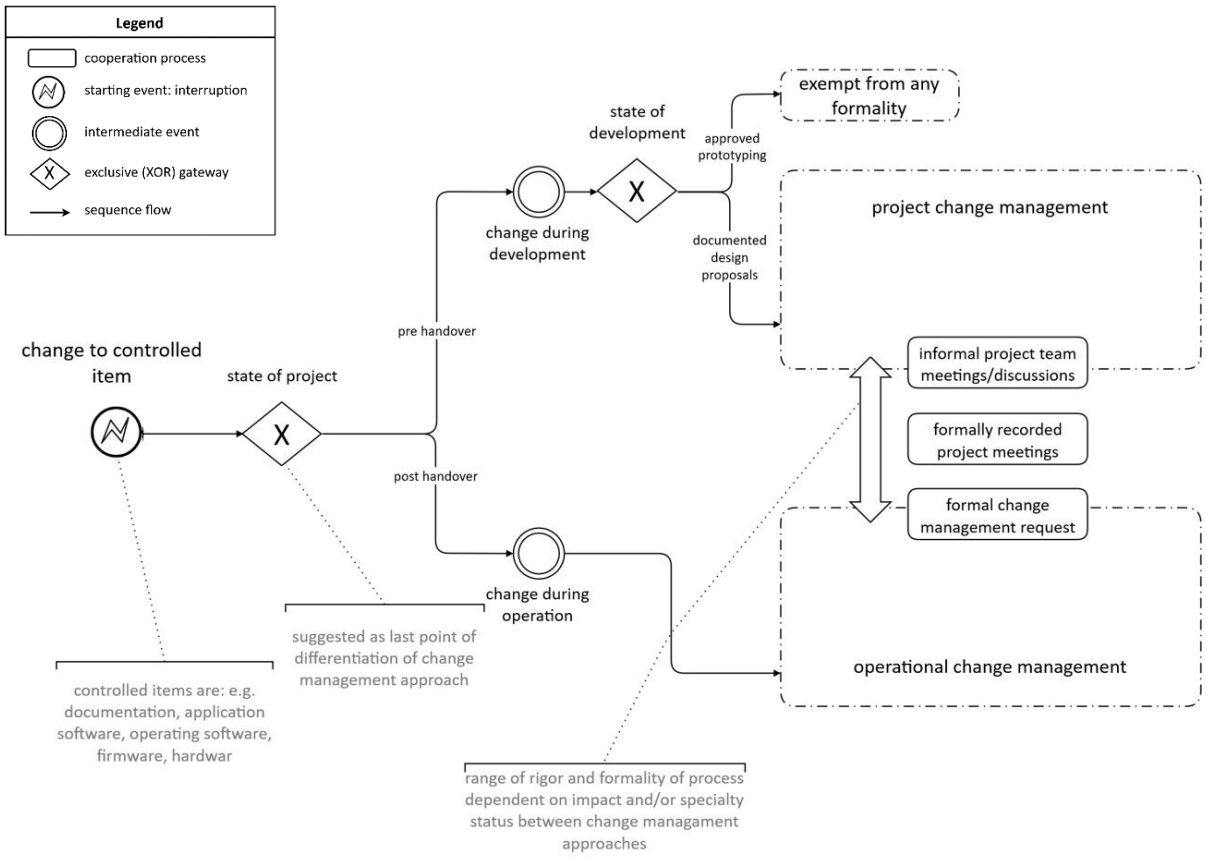

Figure 5: GAMP-suggested workflow as a general overview of rigour and formality; non-conform with BMPN++ standard in exchange for a general overview

5.1.2. Specifics on Project Change Management

As the more informal approach, project change management, is more loosely defined and more freedom of implementation is given to the manufacturer. If the item directly affected by a needed change has been previously approved by the plant user in any form, a formal review and approval of the suggested change is advised. This includes assessing the scope and impact as well as any risk associated with the proposed change. Afterwards which the change can be directly realised and implemented. Any documentation affected should also be revised. After a formal verification, that the aPS is still GMP conform, GAMP 5 also explicitly mentions an approval step by a project manager to then conclude the change as closed. All these process steps, except for the possibility for the customer to raise the proposed change, are solely associated with teams from the plant manufacturer and thus require no inter-

cooperation communication. As GAMP 5 is written such as to be used by both plant suppliers as well as users, it refers to the latter as the “regulated company” because it is within the plant users’ responsibility to be able to verify GMP compliance in the case of an audit.

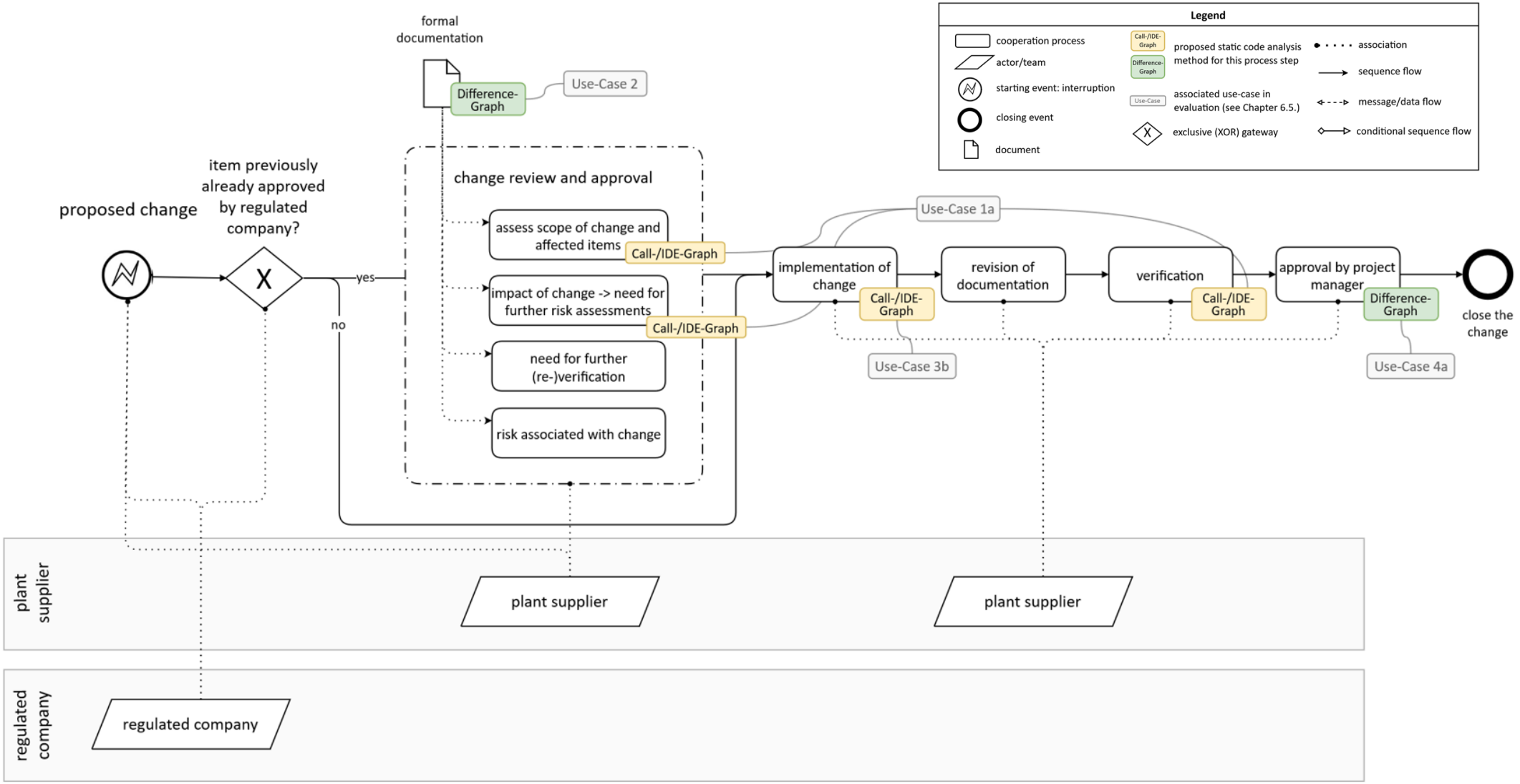

Figure 6: Specific GAMP-suggested workflow for project change management; highlighted: integration of helpful graphs as off the concept in Chapter 5.2.

5.1.3. Specifics on Operational Change Management

As a consequence of the recommended elevation in the formality of change management, GAMP 5 guidelines provide a more comprehensive suggestion of the operational change management workflow. This is particularly evident in the requirement for an explicit "impact and risk assessment" and the potential need for revisions in associated documents. Such risk assessment is imperative for any operational plant in accordance with the risk-based approach outlined in the GMP regulations (see Chapter 2.3.1.). Furthermore, the development and actual implementation of the software changes are separated by firstly commencing with an internal testing phase conducted by the supplier, followed by explicit approval of the new software version by the plant user. In certain instances, this may necessitate additional training for personnel responsible for plant operation and maintenance. The inclusion of the plant user as a pivotal step in this workflow can lead to delays if effective communication between teams from different organisations is not efficiently managed. Such delays can be particularly detrimental, especially as operational downtime is of critical concern. Overall, the ISPE does not specify any specific teams or personnel responsible for each process step but is only concerned with mentioning which company is generally involved in them. The exception is the introduction of a suggestion of a “Quality Unit” at the plant supplier, solely responsible for overseeing that procedures are being followed in accordance with the workflow.

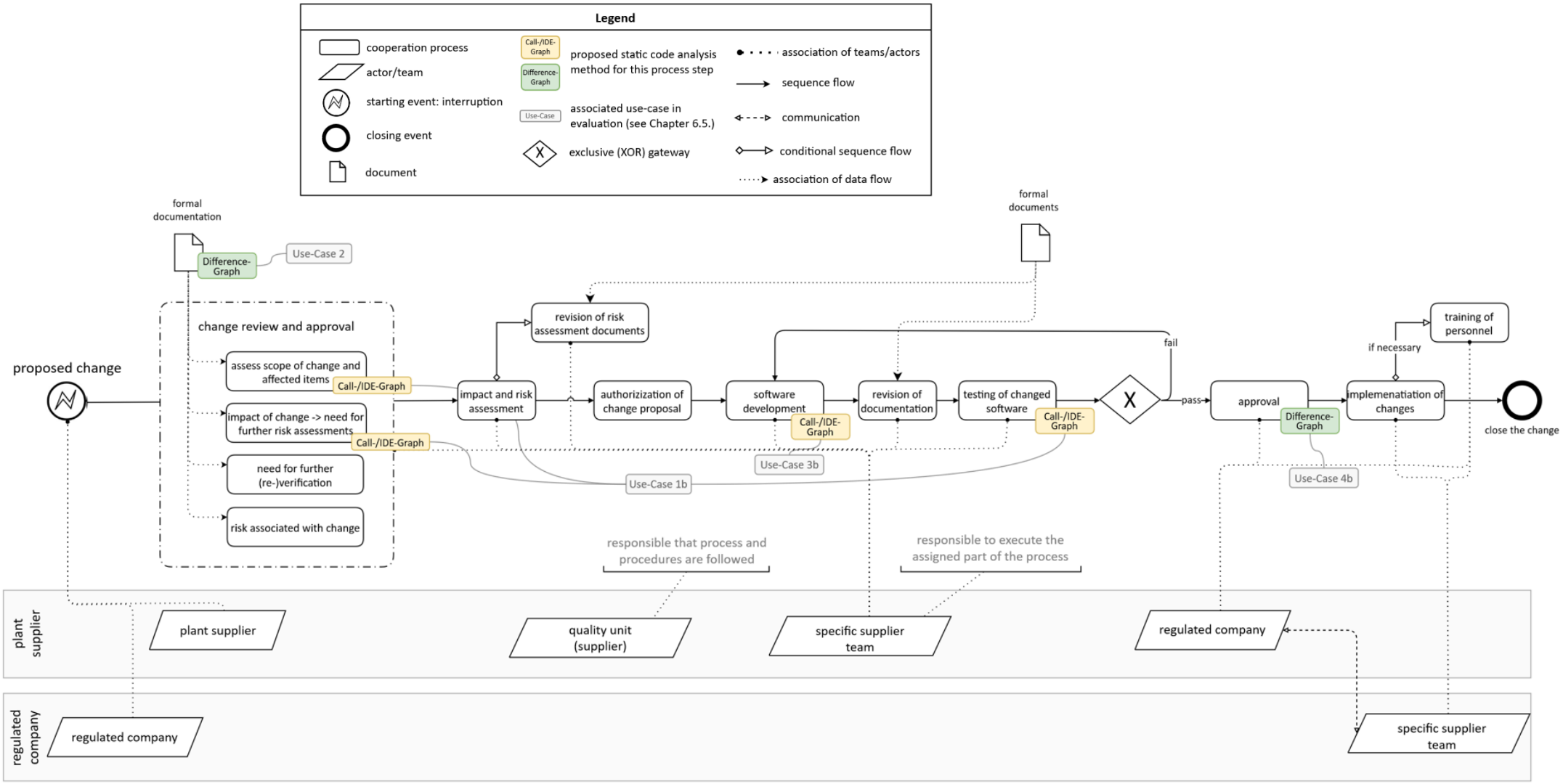

Figure 7: Specific GAMP-suggested workflow for operational change management; highlighted: integration of helpful graphs as off the concept in Chapter 5.2.

5.2. Integrating Static Code Analysis Methods in Critical Steps of a GAMP-Derived Workflow to Determine Change Impact

In the following three graphical methods of static code analysis, *Call*, *IDE*, and *Difference Graph*s, are presented and reviewed in accordance with the Requirements derived in Chapter 3. Furthermore, it is outlined how and where the previously introduced GAMP-suggested workflow benefits from integrating these methods. The specific process steps deemed as benefitting greatly from integrating these graphs are highlighted respectively as such in Figure 6 and Figure 7. Furthermore, by design all these graphs are independent of the underlying programming language in use and in the scope of a prototypical implementation have been shown be able to be created automatically, thereby fulfilling *Requirement 3* and *Requirement 4*.

Call Graph

DDE in PLC software can most easily can be represented using *Call Graphs* [34]. These graphs follow each call within the POUs of the project to form a hierarchical tree of call dependencies. These trees can in the next instance be analysed on their structure using established design patterns [35] to find anomalies like heavily reused “central units”, which are being called by many different POUs throughout the project, or so-called “cornflower POUs”, which are recognisable by their many calls going outwards. This kind of insight gained into the project at a high level by simply visually analysing *Call Graphs* can be used to assess the amount of interdependence and thus helps a-priory gauging impacts of changes e.g., when deciding on how to realise the change proposal most efficiently.

The remarkable versatility of this visual representation of dependencies lies in its capacity to be generated independently of the implementation of changes, rendering it applicable across the entirety of the workflow and project life cycle as of *Requirement 5*. Given that *Call Graphs*, by their definition, render data exchange transparent, they prove particularly valuable during the "change review and approval" process, as well as in any subsequent formal or informal testing or verification procedures, to trace dependencies and determine which other POUs might be affected by the change. Thus, aiding in deciding the extent of the recertification work required. Through the graphical format of *Call Graphs*, one can further assert their utility in supporting documentation and facilitating communication within the plant supplier throughout these process steps as respectively derived in *Requirement 6* and *Requirement 7*. Furthermore, the graphical nature of *Call Graphs* does not require in-depth knowledge in any programming language and can be reasonably assumed to be usable regardless of the stakeholder, as defined in *Requirement 1*, such as in the “approval by project manager” in the project change management workflow.

IDE Graph

Similar to hidden dependencies in change propagation analysis in higher-level languages that need to be taken into consideration to fully comprehend interdependencies [57], IDE through global variables, can be considered to be not as readily apparent as DDE through calls [34]. Hence, to grasp all possible propagations of changes through data exchanges in PLC software, *IDE Graphs*, visualising reads and writes from and to global variables, always need to be considered in conjugation with *Call Graphs*.

As such *IDE Graphs* are of similar graphical nature to *Call Graphs* and are similarly independent of the implementation of changes they can be applied to the same processes. Additionally, integrating *IDE Graphs* to the processes already enhanced by *Call Graphs* as outlined above, yields a more comprehensive overview of all dependencies, in turn improving the decision-making, documentation, or communication in the respective process.

Difference Graph

The distinctive feature of *Difference Graphs* is their possibility to visually highlight the explicit differences between two software versions. This is accomplished by employing color-coding to identify additions, deletions, or internal modifications of POUs within a shared *Call Graph*. In doing so, it inherits all the advantages of the *Call Graph*, and thereby fulfilling the Requirements to the same extent, with its most important attribute being its comprehensibility even without extensive domain expertise. Although, by definition, it is dependent on the change already being realised, it can prove highly valuable in subsequent stages of the process. Especially for approval processes done by external stakeholders, such as the customer, who may not be as close to the development cycle, it can be helpful to retrace the exact changes to gain a better understanding of what requires approval and the rationale behind the changes. Furthermore, it can be argued that any communication required in these approval processes can similarly benefit from this graphical representation by the fact that for both parties this presents an understandable common ground.

6. Introduction to Prototypical Implementation of Static Code Analysis Methods

In addition to manual analysis of the software case-studies conducted in Chapter 7.3., the prototypical implementation for static code analysis *advacode* [58] has been utilized. This chapter gives a brief overview of this code analysis tool and will provide some graphs obtained through it, while the detailed analysis results and their discussion are provided in Chapter 7.3.

Preprocessing steps required

The *advacode* protype has been designed around the analysis of Siemens TIA Portal control software and therefore requires the preprocessing of the metadata of the software through TIA Portal Openness [59] to get Siemens TIA XML files as input. These files contain information about all elements, e.g., POUs, DBs, etc. of the exported software project, as well as their interrelations and dependencies between them through data exchange edges. In turn making *advacode* the ideal fit to realize the trace change propagation, as proposed in the concept in Chapter 5.2. Furthermore, the existence of this tool satisfies *Requirement 3*, as after successful loading of the XML file into *advacode* all structural analysis methods can be created automatically.

Code analysis methods implemented in advacode

From this information various analysis methods are provided through advacode, mainly graphical analysis such as *Call Graphs*, calculation of software metrics, or the identification of duplicate software artifacts. For the concept explicitly the graphical/structural analysis methods have been identified as the most advantageous and useful, as discussed in more detail in Chapter 5.2. This protype provides the possibility to further enhance the *Call Graph* with additional information such as color-coding of the displayed POUs by their main functionality or type (Figure 9). In the *IDE Graph*, within *advacode* referred to as ‘Data Exchange via global DBs’, the read and write edges in the *IDE Graph* display the number of reading or writing variables respectively and thereby provide an overview of the size of interface (Figure 8). Furthermore, the diameter of the POU-nodes can additionally be scaled proportionally to a software metric. Similarly, for the *Difference Graph* the diameter of the changed POUs, color-coded in orange, can be scaled in proportion to the change of a chosen software metric between both versions of the respective POU (Figure 10). A scaling proportional to the change in complexity of a POU can thereby provide a rough insight in the overall size of the change, or a scaling proportional to the change in so-called ‘Fan-In Fan-Out’ in the overall change of the interface of those POUs. These additional settings are mentioned in this chapter for the sake of completeness but are not

regarded in the concept of this thesis, as they usually require a deeper understanding of code analysis to utilize their full potential and thus violate *Requirement 1*. For all of these code analysis methods it is possible to 'Limit to slice here', meaning to only display a selected POU and all its nearest dependencies, as utilized in Figure 8, guaranteeing for the visualizations to be understandable (*Requirement 8*) and manageable to implement into existing documents (*Requirement 7*) independent to the scale of the project (*Requirement 2*). This has been verified in the scope of the expert interviews in Chapter 7.4.

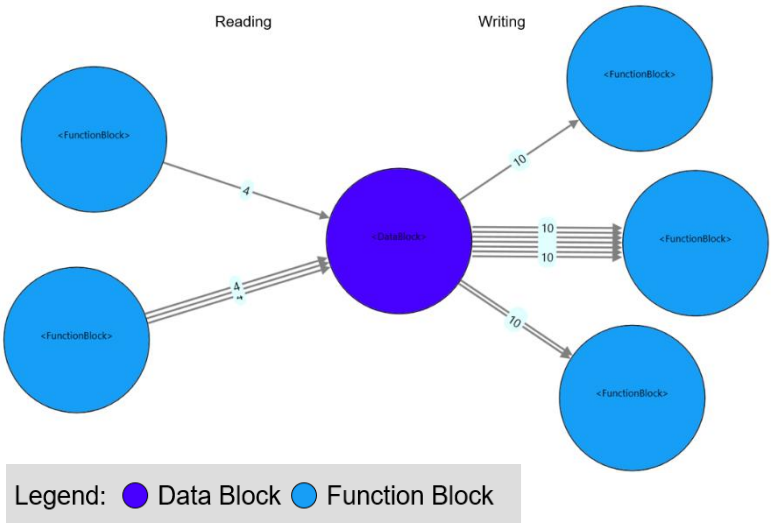

Figure 8: IDE Graph of sub-plant B, sliced to one chosen DB (names of DBs and FBs hidden)

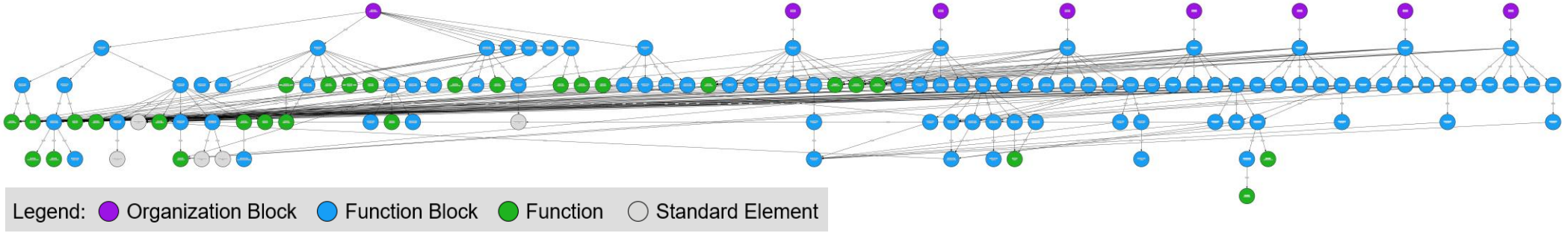

Figure 9: Call Graph of sub-plant B, with color-coding according to POU type

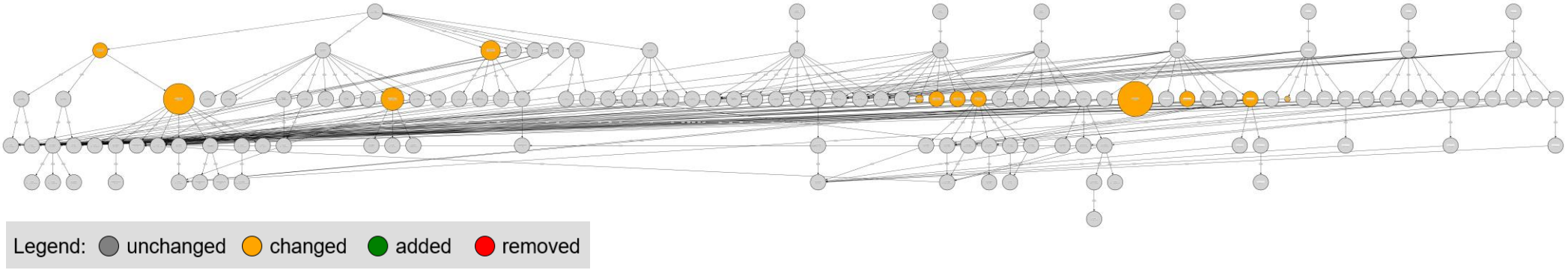

Figure 10: Difference Graph of sub-plant B versions (see Chapter 7.1.1), with scaling of POUs proportional to change of 'FanIn FanOut'-metric.

7. Evaluation of the Concept of Implement Static Code Analysis in the Recertification Workflow

After an introduction of the industry partner, this evaluation is structured into four parts: First, a description of the plant and its software provided by the industry partner, that will serve as a case study for validating the concept outlined in the previous chapter. Secondly, the workflow, deduced from expert interviews at the industry partner, is outlined, and parallels to the GAMP-provided workflow are highlighted. Next up, software changes are being discussed to evaluate methods introduced in the previously elaborated concept. Lastly, an objective evaluation, by interviewing experts on their opinions and gut instincts, will conclude the overall helpfulness of the integration of static code analysis in their workflow.

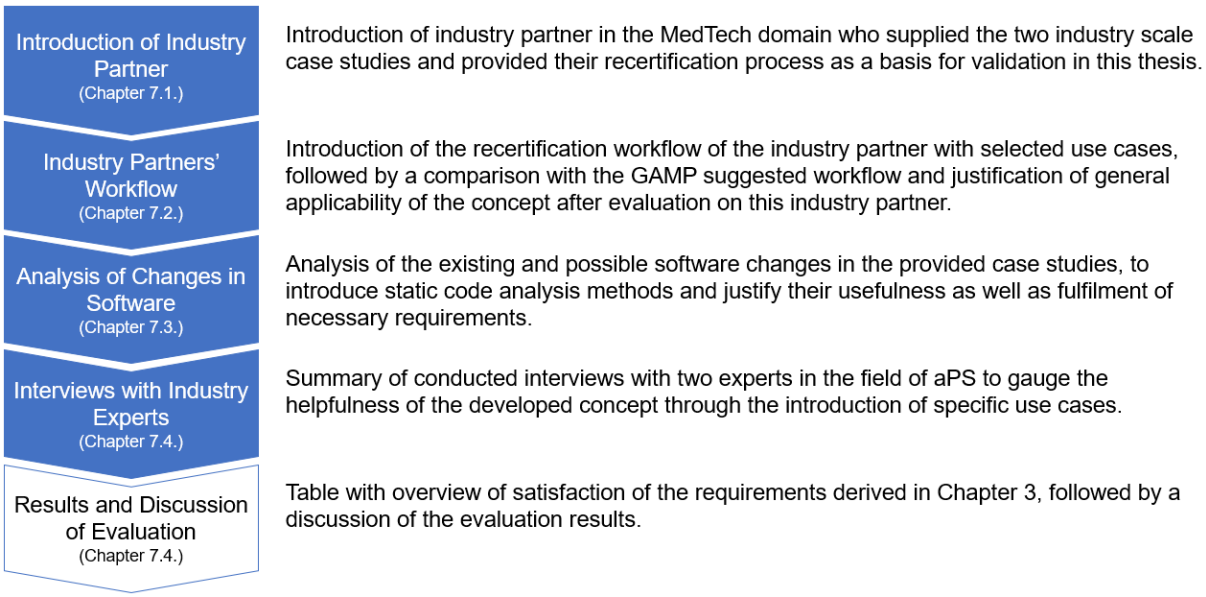

Figure 11: Overview of the evaluation procedure throughout this thesis

7.1. Introduction of Industry Partner

With currently more than 1100 employees across seven production locations worldwide, the industry partner is an established German manufacturer of plants and machinery [60]. Although also present in other fields, for this thesis, particularly the highly flexible automation for the MedTech field is of interest as these highly individualised special purpose plants are especially prone to novice change scenarios and thereby lend themselves nicely for evaluating change impact. The MedTech sector is also a particularly important area of focus for this thesis because of its associated constraints and challenges, as outlined in more detail in Chapter 2.3. In this sector, our industry partner provides a comprehensive suite of services, including assembly and testing of special purpose machinery, as well as training and

ongoing service and support for these systems, including a range of different stakeholders, on which the concept can be evaluated on its requirement to be applicable regardless of the stakeholders involved, as derived in *Requirement 1*.

It is worth noting that the Institute of Automation and Information Systems (AIS) at the Technical University of Munich (TUM) has a long and successful history of collaborating with this particular industry partner on various research projects. As such, the proposal for this thesis was put forward by the AIS, and we were fortunate to have the opportunity to work closely with teamtechnik as an industry partner throughout the research process and gain a deeper understanding of the complexities and nuances of the MedTech sector.

7.1.1 Description of Plant and Affiliated Software of Case Study

The plant at hand is divided into two sub-plants, in the scope of this thesis referred to as *sub-plant A* and *sub-plant B*, each with its own human-machine interface (HMI) and PLC. S*ub-plant A* herby is a circular arrangement of alternating “work modules” with unique, singular processing steps in the assembly of the product and “control modules” that check if the previous processing step has been completed successfully or otherwise mark the product as defective in software. The whole *sub-plant A* envelops twenty of these encapsulated modules and can, for this reason, be considered to be of significant industrial scale, where it is reasonable to assume that further scalability of the concept is possible without significant drawbacks. *Sub-plant B* is placed right after *sub-plant A* and waits for fifteen successfully assembled products to simultaneously place in a plastic tray and then further package these trays within a cardboard box. Although, at first sight, this sub-plant does not appear to have as distinctly separated modules as *sub-plant A*, the modularisation has been achieved in software with seven modules.

Overall, the software is written mainly in LD, but in some parts also in ST and SCL, therefore the proceeding validations and evaluations cover *Requirement 5*, of being independently applicable to the IEC language in use. In both case studies, the software is structured in a modular way, to enhance reusability and sustaining good maintainability, e.g., a clear separation of encapsulated, reusable modules from so-called infrastructure POUs. By definition, this form of keeping good maintainability provides an efficient basis for handling software evolution [61].

7.2. Certification Workflow of the Industry Partner and Comparison with GAMP

The modelled workflows in this chapter abstractly describe the certification process as is followed by the industry partner. Core distinction is whether the change is required before, during, or after the formal

qualification test. After a detailed description to understand the main aspects, the workflow is then compared with the GAMP-suggested workflow as derived in the previous Chapter 5, to justify the viability of evaluating the concept on this specific industry partner while still being generally applicable to other GxP regulated entities.

7.2.1. Description of Industry Partners' Workflow

Two independent workflows can be deducted within the broader range of software change scenarios at the industry partner. One, before any qualification process, when changes are required during the development phase (Figure 12), and the second one, when changes are required during or after the qualification of the aPS (Figure 13). Because these workflows were deduced from interviews with a test engineer, it is predisposed that the issues and to further extend their required changes are being proposed by the test engineer. This assumption has been confirmed by the evaluation interviews elaborated in Chapter 7.4. Furthermore, the abstraction of the depicted workflows in the form of a decision tree (see Appendix) has been confirmed to be correct during these interviews, which in turn deems the BPMN++ models of the industry partner correct too. Although it was commented that this workflow is specific to the provided case-study and the details, especially for the more informal parts of the workflows, are heavily reliant on each customers' requests and desires. Nevertheless, the encompassing workflows and comprehensive evaluations span a broad spectrum of formality, thus affirming the concept's adaptability across an extensive array of use cases and its potential for generalization.

Before qualification

In this case, no rigour formality is called for, and the aim is to go for a more agile approach, where just a simple coordination between test engineer and software engineer is found to be an efficient and sufficient approach. This coordination presents a classic form of informal communication where no documentation of the information exchanged is held (depicted in Figure 12 as a dashed line with arrows on both ends connecting the test and software engineer). Afterwards, the software engineer implements the software change (depicted in Figure 12 as the dotted line connecting the software engineer and the process step "Implementing change in software"). Design specifications or plant hardware can be revised depending on the change performed. The network of actors (Figure 12 on the right-hand side), therefore, only includes the test engineer and software engineer, who are both working at the same location (solid ellipse) and are employed by the same company (dashed ellipse). The fact that both actors are at the same location and company can be taken as reassurance that their communication and cooperation is of good quality, as they have most likely cooperated in the past, are familiar with the same software and tools, as well as hold similar mental models as introduced by Vogel-Heuser et al. [53]. The

test engineers usually work alone or in a team of two, depending on the plant size. A team of two test engineers were delegated to the specific plant used for evaluation in this thesis.

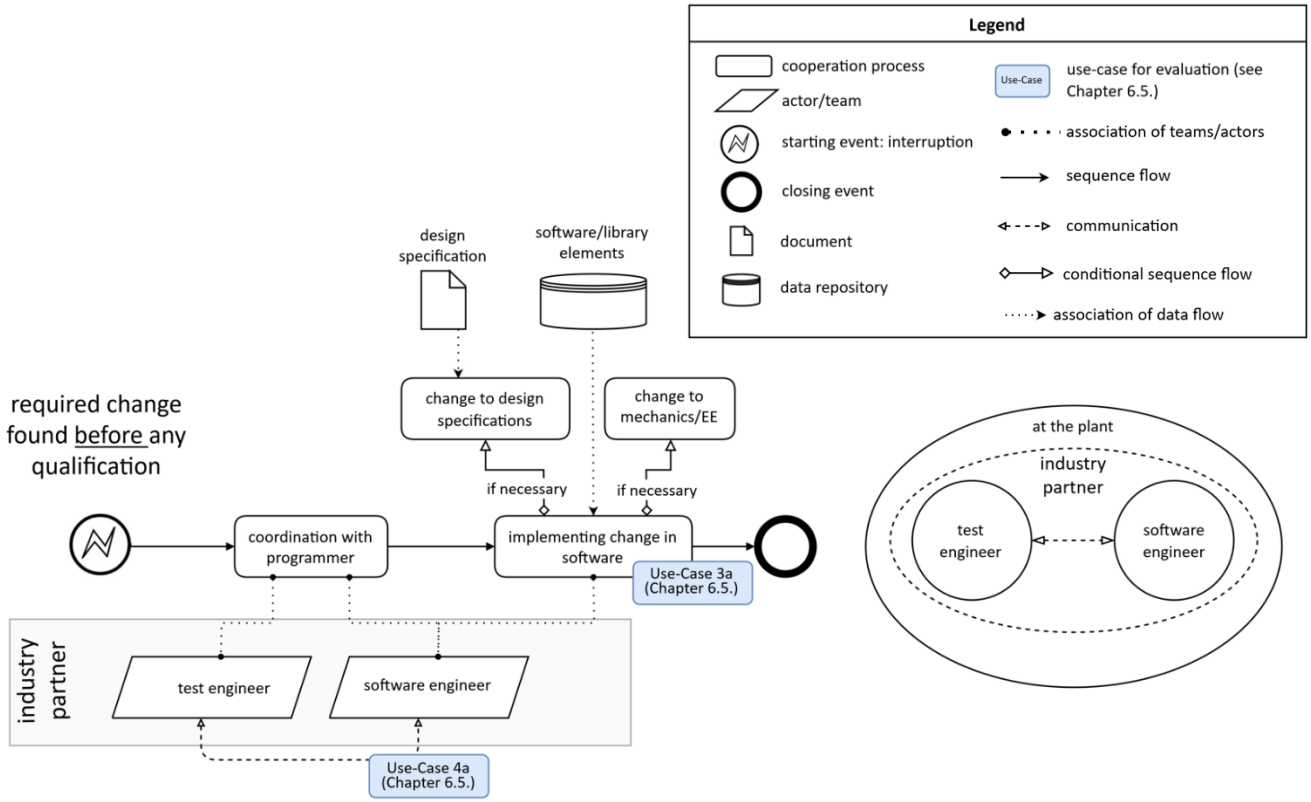

Figure 12: "Cooperation diagram" and corresponding "network of actors" for industry partners' workflow for changes before the qualification; highlighted: use cases as introduced for expert interviews in chapter 7.4.2.

During and after qualification

Firstly, issues that arise during the qualification process of a plant and require software changes to be fixed, must undergo an assessment of their criticality. This form of risk assessment is in accordance with the risk-based approach laid out by the GMP regulations (see Chapter 2.3.1.). Based on the criticality, the risk-assessment team (RA-team) at the plant manufacturer evaluates whether informal coordination with the customer is deemed an efficient and sufficient approach, or whether a formal change request must be filled out on which basis a risk assessment on the proposed change will be done. Both the assessment of criticality and the assessment of the proposed change are documented and explained in a so-called risk-analysis matrix in the form of an Excel table. Similarly, the change request is a formal document filed by the test engineer, most notably including a fault description, a recommended measure for that fault and an estimation of the effect on the system. The two change requests received from other projects are entirely textual, with the only exception being the possibility of an added decision tree to document the conclusion of an alteration in user safety, but not GMP-required product safety. For changes to non-critical software artefacts, the coordination with the customer requires communication

with a quality-control team from the customer's side. As seen in the network of actors (Figure 14) the test engineer, as well as the software engineer, are on-site at the customer for qualification, resulting in much easier communication between these stakeholders. However, they are from vastly different domains and may have no prior experience of working together. Especially the lack of common domain expertise, as present between test engineers and customer teams, can significantly hamper communication efforts, as validated by expert interviews in Chapter 7.4. The implementation of the change into the software follows a similar principle as described in the workflow for changes before qualification above. But depending on the criticality assessed prior, either an informal internal software test is enough to close the change, or a formalised approval by the customer, followed by a formalised qualification process to ensure compliance with GMP, is required. This workflow clearly shows the strength of the proposed concept of aiding the communication between teams from the plant manufacturer and customer, as they need to make agile decisions during time-critical qualification and safety-critical decisions when approving implemented changes. Mainly because a visual representation allows to convey information to other stakeholders without the need for deep domain knowledge, e.g. proficiency in specific programming languages. Although there is no direct communication between the on-site test engineers and the RA-team, they interact with the change request document. As the customer, and thereby the plant, can be located in a different time zone altogether, direct communication is additionally hampered due to the fact, that both stakeholders are at different locations. This requires such documents to be as comprehensible as possible, to avoid the need for direct communication when the workflow breaks down due to misunderstandings. In turn supporting the concept of adding structural visualisation of the software to enhance documentation, as of *Requirement 7*.

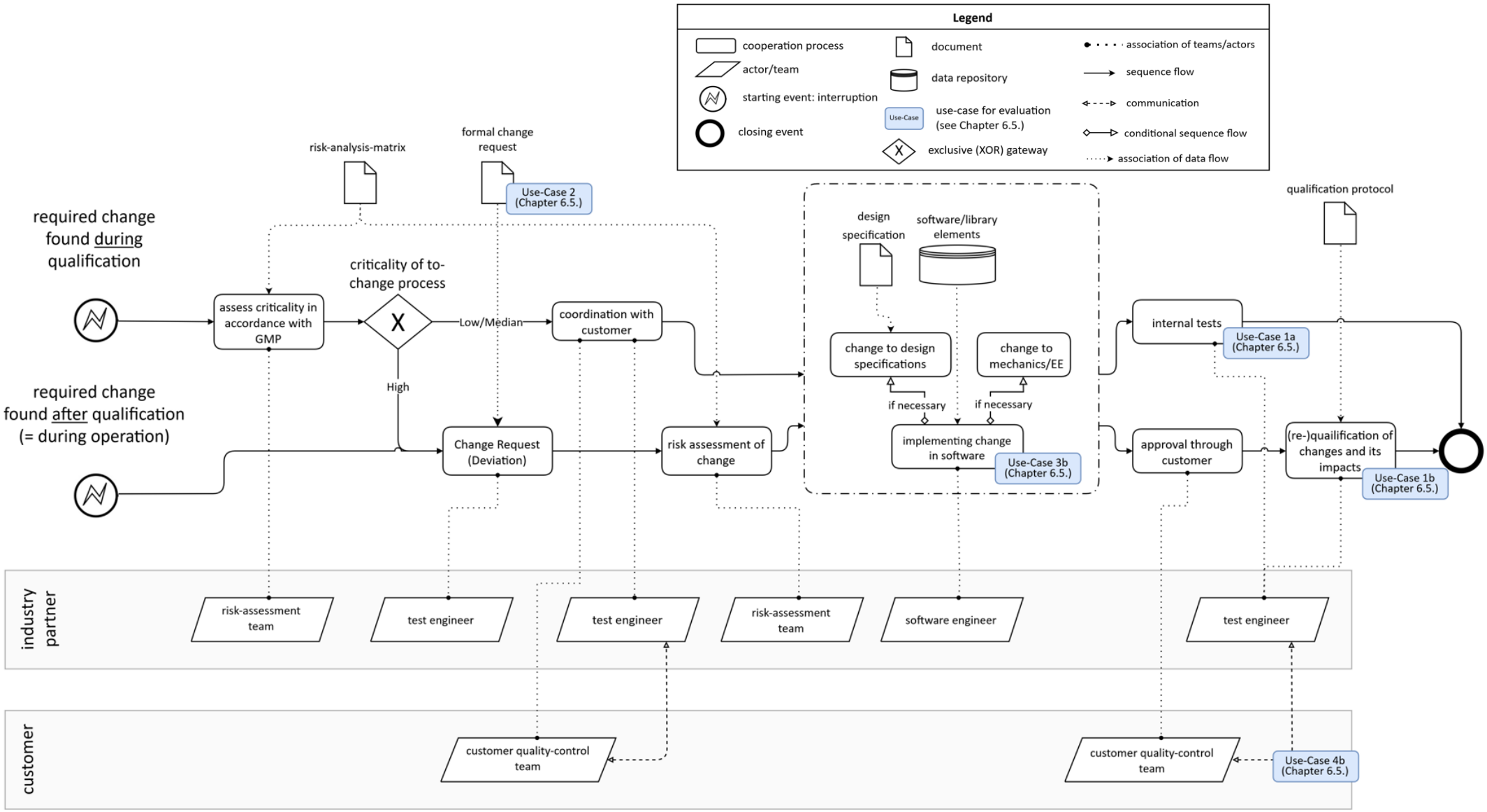

Figure 13: "Cooperation diagram" for the industry partners' workflow for required changes during or after qualification of the plant; highlighted: use cases as introduced for expert interviews in Chapter 7.4.2.

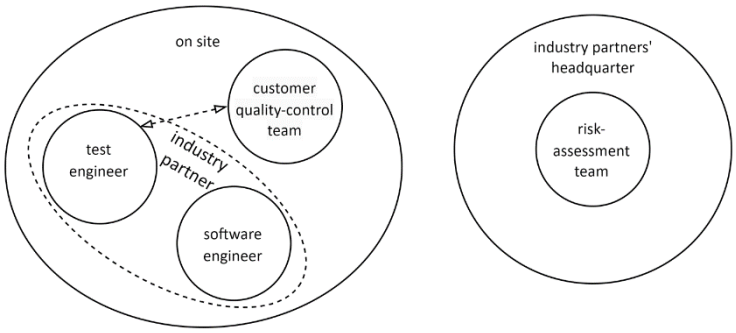

Figure 14: "Network of actors" to the corresponding "cooperation diagram" depicted in Figure 13.

7.2.2. Correlation With GAMP-Introduced Workflow

At first glance the GAMP design philosophy of adapting rigour and formality of the change process in accordance with the respective impact and criticality, is adopted by the industry partners if compared to Figure 5. It manifests mainly by the GAMP suggestion of differentiating the change management process at last at the handover project state, which, for our industry partner hinges on a successful qualification and therefore is the differentiating factor of when a change is required (see entry points in Figure 12 and Figure 13). But this similar approach to adapting rigour and formality also results in both GAMP and industry partners' concluding to three distinct approaches to managing changes with increasing level of rigour and formality (Figure 5: no formality - "project change management" - "operational change management").

Additionally, the more formal workflows show remarkable parallels: Starting with a formal documentation (Figure 13: change request), implementing the risk-based approach through a formal risk assessment process step, all the way to both workflows integrating a formal approval step of the plant user in their respective change management concept. As mentioned in the previous chapter when first introducing the GAMP-suggested workflow, their guideline grows broader and relaxes, the more informal the workflow suggested becomes. This leaves the company implementing its guidelines with more freedom to choose specific process steps. Hence resulting in different approaches for low/median-critical changes during qualification in Figure 12 and project change management in Figure 6.

In Conclusion, as the industry partner follows GAMP suggestions is GMP certified, it implies that the evaluation of the concept within the context of this industry partner can be extrapolated to cover any other company restricted by GxP regulations covered by the GAMP guidelines.

7.3. Analysis of Changes in Industry Partners' Software

This chapter aims to discuss varying change scenarios taken from software evolution as well as expert experience to evaluate the general applicability of the methods of static code analysis introduced in the concept onto real-world industry scale software as described in *Requirement 2*. Additionally, it is reasoned how applicable the static code analysis methods of the concept introduced in Chapter 5.2. are throughout the whole life cycle, as of *Requirement 5*.

7.3.1. Effective Changes Concluded from Empirical Analysis of Two Sub-Plants

Introduction to software versions

The industry partner provided one respective evolutionary software version for each of the sub-plants as a case study and a first basis for applying and evaluating the concept of this thesis. These two case studies only cover the phase just prior to qualification of the plant and hence do not undergo any rigorous validation processes other than internal coordination and ordinary module testing, as concluded from GAMP guidelines (Figure 5) or instructed in the industry partners' own workflow (Figure 12). Still, the analysis gave valuable insight into how changes generally manifest themselves and what software change impacts in later phases can be expected.

Overview of changes to software

Manual analysis resulted in twelve altered POUs both in the *sub-plant B* as well as the substantially more complex *sub-plant A*. These changes affect both infrastructure POUs as well as reusable modules and span far through the call hierarchy of their respective *Call Graphs*, giving a wide range of different change scenarios. As a side effect of the sheer size of the two programs at hand, complete coverage of all software changes cannot be guaranteed without documentation. Yet, in both case studies, changes predominantly manifest themselves in the form of small-scale adaptations to the programs' implementation, defined in one of eight primary change categories by Vogel-Heuser et al. [32].

While not as a comprehensive maturity analysis as proposed by Vogel-Heuser et al. [32], a comparison of the relative evolution of the Halstead complexity in the eleven changed POUs of the *sub-plant B* between both software versions (Figure 15), yields a rough gauge for overall software maturity. The twelfth changed POU only had a variable exchanged and hence does not appear in this illustration as this does not affect Halstead complexity. For the remaining eleven POUs that experienced changes in their implementation, it verifies the advanced stage in software development for both versions, as changes can be summarised as minor enhancements to logic and behaviour and deliberate refactoring aimed at streamlining logical frameworks. Analysis of *sub-plant A* are very similar and help to further

reinforce these findings. Given this validation of software maturity, it is reasonable to conclude that if an evaluation for understandability, as defined in *Requirement 8*, at this software stage, yields positive results, the concept can be assumed to also be understandable in later stages of the life cycle.

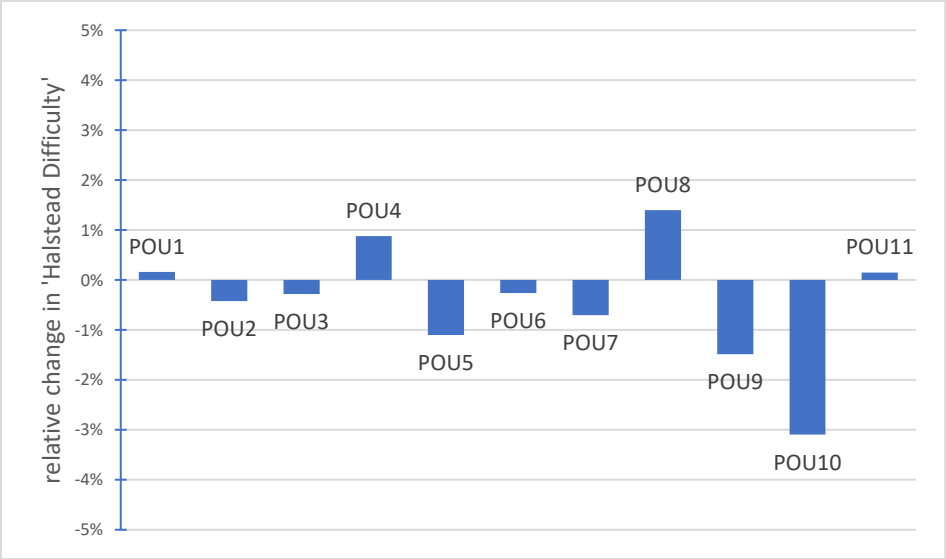

Figure 15: Relative change in Halstead complexity in eleven POUs of sub-plant B between both software versions

Structural and architectural changes

In accordance with the small scale of changes done to the software, in neither case study, added or deleted POUs or hardware elements were found, resulting in no change in overall structure or software architecture. Consequently, small nuances can be picked out from tool-assisted structural analysis methods, like a *Difference Graph*, without overwhelming the user with radical alterations in the presented graph.

Direct propagation of changes – Call Graph

From further structural analysis, it can be concluded that at such late stages of software development, heavily reused POUs, so-called central units, are unlikely to have to undergo changes, as no evolution in these POUs could be determined. It can be concluded that deliberate efforts are made to minimise these changes, primarily because of the extensive reuse of central units, which results in a complex web of interdependencies. This, in turn, leads to a multitude of potential change propagations that both software developers and, by extension, test engineers must be mindful of. This finding is supported by evaluations concluded by Neumann et al. [35], hence further backing the proposal of tool assistance in the form of *Call Graphs* during the software development stage. Time criticality resumes a progressively dominant role as the project advances, and there generally is a concerted effort to minimise downtime during the operational phase [1]. This infers that excessive interdependencies will be avoided during all

phases of software development, in turn presenting *Call Graphs* as helpful tools in all change scenarios to determine possible change impact.

Indirect propagation of changes – global DBs

In the analysis of *sub-plant B*, two cases of new or altered data exchange via such global DBs were found. In the first case, a completely new global DB was created for a variable to be written into, by an already-existing FB. Although in this evolutionary step of the software, no further dependencies on this new DB were found, this change may present new cross-dependencies in a later version. Without any visualisation, the test engineer needs to be informed about this change and always keep this possibility in mind when analysing for later change impact. Furthermore, the *IDE Graph* can present a way to always visualise this new interdependence. The second analysed change introduced a previously non-existent cross-dependency between two FBs by having a changed logic implementation, which required different global variables to be used. Without thoroughly scanning through all variables, this minor addition can easily go unnoticed, leading to possibly missing required recertification. Creating an *IDE Graph* for the analysis that links global DBs with their corresponding read and write vertices for each software version immediately made the impact of both change scenarios apparent. This underscores the utility of *IDE Graphs* throughout the entire workflow.

7.3.2 Possible Changes Throughout the Life Cycle Given by an Expert

During the introductory visit to the industry partner, to get to know the plant at hand, also some change scenarios were being discussed with a test engineer. These scenarios mainly originated from real change requests encountered on similar machinery in the past. Two particular scenarios stood out as especially relevant, as they expand the horizon of real-world changes to the phases during and after the qualification process and additionally cover the different workflows of changes determined to be critical or non-critical (Figure 13).

Firstly, a change scenario, occurring during the qualification process of the assembled plant on-site at the customer, which covers the challenge of preset calibration values, completed in advance. In the example laid out by the test engineer, the plant included a scale for weighing cardboard boxes where the preset calibration limits in the human-machine interface (HMI) were off due to different environmental factors, mainly humidity, at the customers' site. The impact of changing these limits in software on site needed to be quickly assessed to recertify any other possible software artefacts affected by these revised limits. It is reasonable to assume that the limits are defined within the software as constants in a globally accessible DB. An *IDE Graph* providing a comprehensive overview of all POUs reading from this DB and would in this scenario give the user the ability to have a visualisation of possible cross effects of the changed constants. The test engineer categorised this change scenario as non-critical, which, in

accordance with the workflow deduced (Figure 13), only requires an informal agreement with the plant user before implementing the change. This coordination of the test engineer, as a domain expert, and customer, as a presumable non-expert, can be streamlined by presenting the non-expert side with the *IDE Graph*, easily understandably visualising possible cross effects of this change with the read-vertices to the interdependent POUs. This is in accordance with the observations concluded in the description of the industry partners' workflow in Chapter 7.2.1.

The second change scenario, occurring after qualification - meaning during the operational phase of the plant – was the illegitimate entry of negative numbers into the HMI as a specific input. Additionally, this input was deemed to be of high criticality. The challenge was to assess how much of the HMI needed recertification, while keeping downtime to a minimum. This combination of time pressure and formal, rigorous change management means added stress for the test engineer, leading to possible oversights. Here, tool assistance showing possible change propagations, can present a helpful reassurance.

7.4. Industry Expert Interviews on the Helpfulness of the Concept

Another core part to round up the evaluation of the concept introduced in Chapter 5 is to conduct interviews with experts in the field of aPS, which was conducted in the scope of an online workshop. Therefore, a variety of use cases based on the analysis of changes in Chapter 7.3., to apply the concept on, have been chosen to cover the requirements presumed in Chapter 3. This chapter will accordingly firstly introduce the interviewees and their background of expertise, to then introduce the specific use cases chosen and finally conclude the converging or diverging opinions of the experts on the helpfulness of the concept in these use cases.

7.4.1. Introduction of Interview Partners for the Evaluation

In this chapter the evaluation of concept on general helpfulness, based on carefully selected use-cases taken from the industry partners' workflow (Figure 12 and Figure 13) will be summarised. For this industry experts from the industry partner of this thesis, one software developer and one commissioning engineer, have been interviewed. Furthermore, experts from two other companies in the aPS domain were interviewed. This presented valuable insight since it additionally evaluated the usability on different software platforms, namely Siemens TIA and Beckhoff TwinCAT, as well as giving a broader insight into the general helpfulness of the introduction of static code analysis with varying stakeholders (*Requirement 1*). The additional interviewees will be anonymised as *Expert A* and *Expert B* and their background shortly outlined in the following.

Introduction and background of Expert A

At the time of the interview *Expert A* had 2.5 years of experience in software engineering as well as commissioning of aPS, specifically on customer orders of special purpose plants, reflecting a similar domain of aPS to the industry partner introduced in Chapter 7.4.1. Due to the small size of the company, of about 20 employees, he is working for, he has had plenty of customer contact and it presents a noteworthy contribution to inter-team communications. *Expert A* has been mainly trained on Siemens TIA and stated to have never had contact with any form of static code analysis before the interview.

Introduction and background of Expert B

Expert B is at the managerial level of an established platform supplier and has over 15 years of experience with supporting special customer projects in the aPS domain as well as custom in-house research projects. The expertise of *Expert B* mainly lies within the Beckhoff TwinCAT environment, which gave valuable insight into possible future work towards object orientation defined in the 3rd Edition IEC 61131-3, which will be discussed separately in Chapter 8. Furthermore, his position at the management level is key in evaluating *Requirement 1*, applicability regardless of stakeholders.

7.4.2. Procedure of the Interview and Description of Use-Cases

All interviews were conducted separately via online conferences ranging from about 60 to 90 minutes. After a brief introduction of the topic of this thesis, the agenda consisted of firstly a background questionnaire for the interviewees (Appendix C) not part of the industry partner, secondly familiarising the interviewees with the static code analysis in general and further the specific methods applied in the concept of this thesis as of Chapter 5, and finally their subjective evaluation of the helpfulness of the concept in the specific use-cases presented, ranging from 0 (=counterproductive) through 3(=neutral) to 5(=very helpful). The use cases have been taken from scenarios concluded by the analysis of changes presented in Chapter 7.3. and are applied within the context of the industry partners' workflow in Figure 12 and Figure 13 to evaluate their respective methods proposed in Figure 6 and Figure 7. For the interviews of *Expert A* and *Expert B* some of the following use-cases have been generalized, to fit similar scenarios outside of the GMP regulations.

Use-Case 1 – helpfulness in estimating the scale of recertification/-testing needed.

Due to the differentiating approach in rigor and formality suggested by GAMP and also represented in the industry partners' workflow, *use-case 1a* is concerned with required software tests conducted company internally, while *use-case 1b* is concerned with the formal (re-)qualification of changes as seen in Figure 13. Their corresponding code analysis methods presented in the concept in Chapter 5 for the

GAMP-suggested workflow being *Call* and *IDE Graphs* respectively for the "verification"-process step in Figure 6 and "testing of changed software" in Figure 7. As *Expert A* and *Expert B* need not comply with GMP regulations, they do not have as strict of a separation in their change management workflow. In turn, for these interviews both use cases were generalised into helpfulness for software tests conducted during the commissioning phase for *use-case 1a* and during the operational phase for *use-case 1b*. This generalisation makes sure to still evaluate *Requirement 5*, applicability throughout the complete life cycle, while still representing the respective process steps in the industry partners' workflow presented in Figure 13.

Use-Case 2 – helpfulness in documenting software changes.

As formal documentation is only needed for critical software changes, or changes conducted in the later phases of the production system, there is no need to differentiate this use case. Although outside of GMP regulations no formal documentation is required, both *Expert A* and *Expert B* stated to be using some form of version management to track software changes. Thus, these interviewees were presented with the question on their impression of how helpful static code analysis methods would be in a documented form, for tracking and justifying software changes, in turn evaluating the concept on *Requirement 7*. For the interview with the industry partners' experts the use-case was aimed at enhancing just formal documentations.

Use-Case 3 – helpfulness in the decision-making process when implementing software changes.

Similar to *use-case 1*, the evaluation of the helpfulness of the concept in the decision-making process when implementing software changes, has been separated into *use-case 3a* in Figure 12, covering the case for required changes before any qualification, in turn generalised for the interviewees *A* and *B* as the development phase of the aPS. And *use-case 3b* in Figure 13, covering the workflow for required changes during and after qualification, in turn generalised for these interviewees as the commissioning and operational phase. This again assures to cover software changes appearing in any phase of the life cycles as by *Requirement 5*.

Use-Case 4 – helpfulness in enhancing communication.

Herein, *use-case 4a* aims to evaluate the helpfulness of the concept in enhancing informal communication between interdisciplinary teams of the same company. Specifically informal communication in the early stages of the project, as the case in the "coordination between programmer in test engineer" in Figure 12. Whereas *use-case 4b* is concerned with the communication with non-domain experts, such as a customer as occurring during the "approval"-process in the industry partners workflow in Figure 13. The *use-cases 4b* correlates with the "approval"-processes suggested by GAMP

with either a project manager as presented in Figure 6 or a customer team in Figure 7. Overall, both use cases were chosen and presented to evaluate the concept on *Requirement 1*, to be applicable regardless of the stakeholders involved, as well as proving *Requirement 6*, to be also applicable to communication.

7.4.3. Results of the Expert Interviews

Throughout the interview all four experts expressed easy comprehensibility of the graphs presented without any of them having prior in-depth knowledge of static code analysis methods, validating the fulfilment of *Requirement 8*. As the graphs shown in the presentation of the interviews were taken from real-world industrial-scale samples (similar to Figure 8, Figure 9, Figure 10), it is also reasonable to assume a general fulfilment of *Requirement 2*.

Concerning Use-Case 1

The industry partners' experts, particularly the commissioning engineer, stated a general helpfulness of the presented concept, when concerned with estimating the extend of recertification. Especially for generally confirming his suspicions of possible change impact and for complex change scenarios, e.g., changes to the sequential working process of the plant. Additionally, the helpfulness heavily depends on the module directly affected by the change, as the commission engineer stated, that for well-known reused 'standard modules' such a confirmation might not be as necessary. Non the less, even for such change scenarios it presents no disadvantage because of an automatic generation (*Requirement 3*) and the user can freely decide how heavily to rely on the presented graphs on a case-by-case basis. *Expert B* agreed that implementing static code analysis methods, such as *Call-* and *IDE Graphs* into the testing/verification process can be very helpful in assessing the scope of change impact throughout the whole life cycle, despite the testing processes not being as rigour as in companies underlying GMP regulations. From this assessment, it can be reasonably concluded that the concept will be similarly, if not even more helpful for formal, more rigorous, testing/verification processes. As *Expert A* uses a digital twin (DT) to conduct all software testing, this use-case was not applicable, as DTs are a virtual representation of the aPS at hand to e.g., automatically run tests and simulations on [62]. However, *Expert A* agreed on the good understandability of the presented graphs, even on an industrial scale software. Additionally *Expert A* pointed out the extraordinary time taken up by designing a new DT for every special purpose plant, limiting its applicability in the context of this thesis as it contradicts *Requirement 3.*

Concerning Use-Case 2

The commissioning engineer expressed extraordinary delight when presented with *use-case 2*, of implementing graphical representations into the documents, explicitly their 'Change Request' document

(Figure 13), supporting a great fulfilment of *Requirement 7*. Also, both *Expert A* and *Expert B* agreed that *Call*, *IDE*, as well as *Difference Graphs* present themselves as helpful and very helpful tools to assist in documenting software changes due to their exceptional interdisciplinary understandability. Although *Expert A*, pointed out that their current, more informal, version management system is subjectively 'enough' and feared that implementation of a time-consuming documentation step could present itself as challenging, further supporting the need for *Requirement 3*.

Concerning Use-Case 3

The industry partners' experts tended to favour the implementation of code analysis in earlier stages of software development such as *use-case 3a*. Additionally the high-level overview was positively pointed out, as it was envisioned to be too tedious to have to look through at a more detailed representation. On the other hand, the evaluation of *Expert A* and *B* diverged on the helpfulness of the concept for applying to the decision-making process, with *Expert A* claiming great helpfulness for deciding on how to implement a software change during the commissioning and operational phase. Primarily because of prior instances where swift bug fixes led to unanticipated issues arising from undisclosed dependencies, while stating that during the development phase, their software structure would lack the necessary reliability for implementing *Call-* or *IDE Graphs*, and they anticipated frequent changes, potentially making prior assumptions about dependencies obsolete. On the other hand, *Expert B* expressed exceptional helpfulness for the implementation of Dependency Graphs in the decision-making process during the development and commissioning phase. However, this support dwindled during the operational phase, as experience had shown that changes during this phase would usually be too minor to have any significant impact. The varying perspectives on the utility of this concept, depending on the particular phase of the aPS, can be attributed to the considerable differences in the sizes of the companies represented by both experts. During the interview with *Expert A*, it became evident that they adopt a more agile approach to software change management compared to the company represented by *Expert B*. Consequently, *Expert A's* company tends to encounter significant software changes even in the later stages of development. As any companies in the MedTech domain must be coherent with GMP, they must have a change management workflow with some rigour and formality, thus reducing agility and preventing to major changes in later phases to some extent. In turn, for the scope of this thesis it can concluded that the concept is very helpful for *use-case 3a*, while helpfulness depends on the exact stage of the software for *use-case 3b*.

Concerning Use-Case 4

The commissioning engineering stated to have a lot of contact with the customer, especially when commissioning the plant at the customer site. In terms of helpfulness of potentially adding *Difference*

Graphs into this communication, he differentiated between customer teams that have knowledge in software development and such that do not. As in the MedTech domain most customers are very specialized on the medical side, he mentioned that they might not be interested in such representation as it is not strictly necessary for the specific *use-case 4b* Figure 13 and might even feel a bit overwhelmed, questioning the fulfilment of *Requirement 1*. Similarly to *Expert A*, the teams working on one sub-plant are usually of smaller size, in turn making the implementation of static code analysis in scenarios like *use-case 4a* not necessarily advantageous. Although he mentioned that they can be quite helpful if changes need to be escalated to higher, managerial, levels. Both additional experts agreed on the exceptional helpfulness of *Difference Graphs* to communicate software changes with the customer because of their easy comprehensibility, as defined in *Requirement 8*. Especially *Expert B* pointed out a possibly great usefulness in implementing such graphs in team meetings with him, as a project manager, present as he might not be as involved in the intricacies of the software of an aPS and still being able to gain a comprehensive overview to engage in the discussions. For *use-case 4a*, the opinions of both experts again diverged because of their respective company sizes. *Expert A* pointed out that in their context, where typically only two or three software engineers are working at the same facility, both engineers are deeply immersed in the entire software system. As a result, the current informal method of communicating changes is deemed sufficient, and any potentially time-consuming efforts to improve information exchange might not be embraced by the users. On the other hand, *Expert B* expressed potentially great helpfulness in also implementing these methods in inter-team communications such as *use-case 4a* as teams are usually significantly bigger. In conclusion, the implementation of the concept onto an existing workflow of informal communication needs to be evaluated carefully for smaller teams, as not to impede any existing established exchange, while for bigger teams it can be assumed to be generally helpful. It's worth highlighting a potential concern raised by Expert B regarding the use of *Call* and *Difference Graphs* for improving communication with customers, e.g., for justifying the costs of customer change requests, as these graphs could potentially pose a security risk by exposing a significant amount of know-how through the revelation of the complete software structure. A solution would be to only present subgraphs necessary, but this has to be evaluated on a case-by-case basis to ensure that the subgraph still conveys enough information as to actually enhance the communication.

7.5. Results of Evaluation and Discussion

Table 1 presents a comprehensive overview of all requirements derived in Chapter 3 with their associated use cases used in the conducted expert interviews and ultimately a grading of the fulfilment of each requirement as concluded from these interviews with a brief explanation.

Requirements		Associated Use-Case(s)	Result of Evaluation
Req. 1	Applicability regardless of stakeholders	use-cases 2 and 4	4/5 generally helpful with some exceptions for non-technical customers
Req. 2	Scalability to industrial scale	all use-cases	5/5 generally good scalable
Req. 3	Automatic creation	-	satisfied by implementation (see Chapter 6)
Req. 4	Independence from the programming language	-	satisfied by nature of chosen analysis methods (see Chapter 5.2.)
Req. 5	Applicability throughout the life cycle	use-cases 1 and 3	3-5/5 dependant on size of change (generally more helpful for complex changes) and user dependant on perceived helpfulness at specific points in life cycle phases
Req. 6	Support process-steps and communication	use-case 4	4/5 intra-company communications: very helpful for bigger teams and different company hierarchies; case-by-case dependence for small teams with existing informal communication 0 or 4-5/5 inter-company communications: case-by-case dependant on the known background of the customer, also potential threat to validity when concerned with know-how protection
Req. 7	Useable for documentation	use-case 2	5/5 generally very helpful for formal documentation of software changes
Req. 8	Presented in an understandable way	all	5/5 generally easy to comprehend without in-depth knowledge

Table 1: Summary of Requirements and their grade of fulfilment concluded from expert interviews

It is noteworthy to mention, that the validation of the concept on the case studies in Chapter 7.3. as well as all expert interviews showed comprehensive fulfilment of the scalability (*Requirement 2*) as well as understandability (*Requirement 8*) of the developed concept. Also, the implementation of graphical representations of static code analysis methods into documentation (*Requirement 7*), also outside of GMP regulations, were evaluated to be very helpful throughout all experts. Although the analysis of the case studies conducted in Chapter 7.3. showed no limitations of applicability of the concept through the life cycle of aPS (*Requirement 5*), the industry experts varied in their assessment, of when it is of most

assistance. This can possibly be attributed to different approaches to software development and certainly warrants further research and questioning of experts, to possibly further enhance this concept. But it should be noted that the concept in no point has been deemed to be a hurdle or even counterproductive, hence leaving the user to decide on his own when exactly to resort to assistance through the static code analysis methods. Also, the assessment of the experts, that the helpfulness of the concept heavily relies on the scale of change at hand, is logical. The remark of *Expert B* about possibly revealing company sensitive information by providing a complete *Call Graph* to a customer is a new insight and has not been brought up in any current research work of static code analysis methods as presented in Chapter 4.1. This thread can be possibly mitigated by only revealing sub-graphs, but then the overall usefulness has to be newly assessed. Other than this unique possibility, the concept has been deemed to be overall helpful in enhancing communication both within the company as well as with external teams (*Requirement 6*).

8. Conclusion and Outlook

This concluding chapter will provide a comprehensive summary of the research conducted in this thesis and finally will present an outlook for potential future research within this specific domain.

8.1. Conclusion of Research

In the context of this thesis, an innovative concept has been developed for the integration of static code analysis methods into the recertification workflows of GxP-regulated companies such as the MedTech domain. Initially, specific processes and communications within the recertification workflow, as outlined in the GAMP 5 guidelines, were identified as potentially benefiting from the use of code analysis methods. Required changes typically trigger these recertification processes. To assess the potential impact of these changes, *Call* and *IDE Graphs* have been introduced, providing users with a comprehensive view of all data exchanges between POUs. Additionally, the concept of *Difference Graphs* has been introduced to enhance communication and documentation of changes after their occurrence. The implementation of these analysis methods within a generalised workflow was subsequently compared to a specific recertification workflow in the MedTech industry. The strength of this thesis lies in its capacity to validate the concept's applicability by examining two industrial-scale case studies. The final evaluation was conducted by experts from various backgrounds within the aPS domain and confirmed the overall helpfulness of the concept in practical use cases.

8.2. Outlook for Further Research Projects

To achieve a comprehensive validation of the concept across the entire life cycle, it would be imperative to secure additional software versions corresponding to different phases of the plant. Additionally, to these software versions, the associated formal change request documents could be used to propose a potential assessment of the concept's precision and accuracy in presenting possible change propagations to the user. Moreover, as alluded to briefly in Chapter 7.4, it was pointed out by *Expert B* that the latest iteration, the 3rd Edition, of the IEC 61131-3 introduces object-oriented features. Due the limitation of the implementation only being able to analyse Siemens TIA projects as described in Chapter 6, and the lack for support for object orientation within the Siemens environment, object orientation has not been considered within the scope of this thesis. Furthermore because of the industry's slow embrace of this extension the concept presented in this thesis remains relevant for a multitude of scenarios. However, to ensure its future applicability, it would be prudent to closely examine and assess the ramifications of the emerging object-oriented forms of data exchange, specifically methods, inheritance, and interfaces. Such an exploration may yield novel ways to utilise UML diagrams in the proposed concept.

References

[1] B. Vogel-Heuser, A. Fay, I. Schaefer, and M. Tichy, “Evolution of software in automated production systems: Challenges and research directions,” *Journal of Systems and Software*, vol. 110, pp. 54–84, 2015, doi: 10.1016/j.jss.2015.08.026.

[2] “EudraLex - The Rules Governing Medicinal Products in the European Union - Volume 4 - EU Guidelines for Good Manufacturing Practice for Medicinal Products for Human and Veterinary Use - Part 1 Chapter 5: Production,”

[3] E. Westkämper, “Adaptable Production Structures,” in *Manufacturing Technologies for Machines of the Future*: Springer, Berlin, Heidelberg, 2003, pp. 87–120. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-642-55776-7_4

[4] D. C. Mcfarlane and S. Bussmann, “Developments in holonic production planning and control,” *Production Planning & Control*, vol. 11, no. 6, pp. 522–536, 2000, doi: 10.1080/095372800414089.

[5] Birkhofer,R.,Feldmeier,G.,Kalhoff,J.,Kleedörfer,C.,Leidner,M.,Mildenberger, R.,Mühlhause,M.,Niemann,J.,Schrieber,R.,Wickinger,J.,Winzenick,M., Wollschläger,M., *Life-cycle-management für produkte und systeme der automation: Ein leitfaden des arbeitskreises systemaspekte im ZVEI fachverband automation*, 2010.

[6] International Electrotechnical Commission, *IEC 61131-3 - Programmable controllers - Part 3: Programming languages*.

[7] Siemens, *Normerfüllung nach IEC 61131-3.* [Online]. Available: https://support.industry.siemens.com/cs/document/8790932/iec-61131-3-and-simatic-s7?dti=0&lc=en-DE

[8] B. Vogel-Heuser, J. Fischer, S. Rosch, S. Feldmann, and S. Ulewicz, “Challenges for maintenance of PLC-software and its related hardware for automated production systems: Selected industrial Case Studies,” in *2015 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, Bremen, Germany, 2015, pp. 362–371.

[9] B. Vogel-Heuser *et al.,* “Interdisciplinary engineering of cyber-physical production systems: highlighting the benefits of a combined interdisciplinary modelling approach on the basis of an industrial case,” *Des. Sci.*, vol. 6, 2020, doi: 10.1017/dsj.2020.2.

[10] S. F. Langer and U. Lindemann, “Managing Cycles in Development Processes - Analysis and Classification of External Context Factors,” *DS 58-1: Proceedings of ICED 09, the 17th International Conference on Engineering Design, Vol. 1, Design Processes, Palo Alto, CA, USA,*

24.-27.08.2009, pp. 539–550, 2009. [Online]. Available: https://www.designsociety.org/publication/28561/managing+cycles+in+development+processes+-+analysis+and+classification+of+external+context+factors

[11] B. Vogel-Heuser, T. Simon, J. Folmer, R. Heinrich, K. Rostami, and R. Reussner, "Towards a common classification of changes for information and automated production systems as precondition for maintenance effort estimation," in *2016 IEEE 14th International Conference on Industrial Informatics (INDIN)*, Poitiers, France, Jul. 2016 - Jul. 2016, pp. 166–172.

[12] F. Li, G. Bayrak, K. Kernschmidt, and B. Vogel-Heuser, "Specification of the Requirements to Support Information Technology-Cycles in the Machine and Plant Manufacturing Industry," *IFAC Proceedings Volumes*, vol. 45, no. 6, pp. 1077–1082, 2012, doi: 10.3182/20120523-3-RO-2023.00146.

[13] J. Buckley, T. Mens, M. Zenger, A. Rashid, and G. Kniesel, "Towards a taxonomy of software change," *J. Softw. Maint. Evol.: Res. Pract.*, vol. 17, no. 5, pp. 309–332, 2005, doi: 10.1002/smr.319.

[14] Office Journal of the European Union, "COMMISSION DIRECTIVE (EU) 2017/ 1572 of 15 September 2017 - supplementing Directive 2001/83/EC of the European Parliament and of the Council as regards the principles and guidelines of good manufacturing practice for medicinal products for human use,"

[15] European Commission, "EudraLex - The Rules Governing Medicinal Products in the European Union - Volume 4 - Good Manufacturing Practice Medicinal Products for Human and Veterinary Use - Annex 11: Computerised Systems.,"

[16] "EudraLex - The Rules Governing Medicinal Products in the European Union - Volume 4 - EU Guidelines for Good Manufacturing Practice for Medicinal Products for Human and Veterinary Use - Chapter 1: Pharmaceutical Quality System,"

[17] "EudraLex - Volume 4 - EU Guidelines for Good Manufacturing Practice for Medicinal Products for Human and Veterinary Use - Annex 15: Qualification and Validation,"

[18] *GAMP 5: A risk-based approach to compliant GxP computerized systems*. Tampa, FL: International Society of Pharmaceutical Engineers, 2008.

[19] T. E. White and L. Fischer, Eds., *New Tools for New Times: The Workflow Paradigm: The impact of Information Technology on Business Process Reengineering*: Future Strategies Inc, 1994.

[20] D. Winkler and S. Biffl, *Improving Quality Assurance in Automation Systems Development Projects*. Implementation of CVR: INTECH Open Access Publisher, 2012.

[21] B. Vogel-Heuser, S. Rosch, A. Martini, and M. Tichy, “Technical debt in Automated Production Systems,” in *2015 IEEE 7th International Workshop on Managing Technical Debt (MTD)*, Bremen, Germany, 2015, pp. 49–52.

[22] B. Vogel-Heuser and D. Friedrich, “Integrated automation engineering along the life-cycle,” in *IECON 2002: Proceedings of the 28th annual conference of the IEEE Industrial Electronics Society*, Sevilla, Spain, Nov. 2002, pp. 2473–2478.

[23] B. Vogel-Heuser, J. Folmer, and C. Legat, “Anforderungen an die Softwareevolution in der Automatisierung des Maschinen- und Anlagenbaus,” *at – Automatisierungstechnik*, vol. 62, no. 3, pp. 163–174, 2014, doi: 10.1515/auto-2013-1051.

[24] P. Emanuelsson and U. Nilsson, “A Comparative Study of Industrial Static Analysis Tools,” *Electronic Notes in Theoretical Computer Science*, vol. 217, pp. 5–21, 2008, doi: 10.1016/j.entcs.2008.06.039.

[25] B. Vogel-Heuser and F. Ocker, “Maintainability and evolvability of control software in machine and plant manufacturing — An industrial survey,” *Control Engineering Practice*, vol. 80, pp. 157–173, 2018, doi: 10.1016/j.conengprac.2018.08.007.

[26] *Modularity and architecture of PLC-based software for automated production Systems: An analysis in industrial companies*, 2017. [Online]. Available: https://www.sciencedirect.com/science/article/pii/s0164121217300985

[27] R. Reussner, M. Goedicke, W. Hasselbring, B. Vogel-Heuser, J. Keim, and L. Märtin, *Managed Software Evolution*. Cham: Springer International Publishing, 2019. [Online]. Available: https://library.oapen.org/handle/20.500.12657/22886

[28] E. M. Neumann *et al.,* “Metric-based Identification of Target Conflicts in the Development of Industrial Automation Software Libraries,” in *2022 IEEE International Conference on Industrial Engineering and Engineering Management (IEEM)*, 2022.

[29] *ISO/IEC 25010.* [Online]. Available: https://iso25000.com/index.php/en/iso-25000-standards/iso-25010 (accessed: 10.2023).

[30] T. J. McCabe, “A Complexity Measure,” *IIEEE Trans. Software Eng.*, SE-2, no. 4, pp. 308–320, 1976, doi: 10.1109/tse.1976.233837.

[31] J. Wilch *et al.,* “Introduction and Evaluation of Complexity Metrics for Network-based, Graphical IEC 61131-3 Programming Languages,” in *IECON 2019 - 45th Annual Conference of the IEEE Industrial Electronics Society*, 2019.

[32] B. Vogel-Heuser, J. Fischer, E.-M. Neumann, and S. Diehm, “Key maturity indicators for module libraries for PLC-based control software in the domain of automated Production Systems,” *IFAC-PapersOnLine*, vol. 51, no. 11, pp. 1610–1617, 2018, doi: 10.1016/j.ifacol.2018.08.261.

[33] E.-M. Neumann, B. Vogel-Heuser, J. Fischer, S. Diehm, M. Schwarz, and T. Englert, “Automation software architectures in automated production systems: an industrial case study in the packaging machine industry,” *Prod. Eng. Res. Devel.*, vol. 16, no. 6, pp. 847–856, 2022, doi: 10.1007/s11740-022-01133-y.

[34] J. Fuchs, S. Feldmann, C. Legat, and B. Vogel-Heuser, “Identification of Design Patterns for IEC 61131-3 in Machine and Plant Manufacturing,” *IFAC Proceedings Volumes*, vol. 47, no. 3, pp. 6092–6097, 2014, doi: 10.3182/20140824-6-ZA-1003.01595.

[35] E.-M. Neumann, B. Vogel-Heuser, J. Fischer, F. Ocker, S. Diehm, and M. Schwarz, “Formalization of Design Patterns and Their Automatic Identification in PLC Software for Architecture Assessment,” *IFAC-PapersOnLine*, vol. 53, no. 2, pp. 7819–7826, 2020, doi: 10.1016/j.ifacol.2020.12.1881.

[36] *Kamp: Karlsruhe architectural maintainability prediction*, 2009. [Online]. Available: http://ceur-ws.org/vol-537/d4f2009_paper08.pdf

[37] *Karlsruhe Architectural Maintainability Prediction (KAMP).* [Online]. Available: https://github.com/KAMP-Research/KAMP (accessed: Jul. 14 2023).

[38] K. Rostami, J. Stammel, R. Heinrich, and R. Reussner, “Architecture-based Assessment and Planning of Change Requests,” in *Proceedings of the 11th International ACM SIGSOFT Conference on Quality of Software Architectures*, Montréal QC Canada, 2015, pp. 21–30.

[39] B. Vogel-Heuser *et al.,* “Maintenance effort estimation with KAMP4aPS for cross-disciplinary automated PLC-based Production Systems - a collaborative approach,” *IFAC-PapersOnLine*, vol. 50, no. 1, pp. 4360–4367, 2017, doi: 10.1016/j.ifacol.2017.08.877.

[40] S. Koch, “Automatische Vorhersage von Änderungsausbreitungen am Beispiel von Automatisierungssystemen,” Karlsruher Institut für Technologie (KIT), 2017.

[41] R. Heinrich, S. Koch, S. Cha, K. Busch, R. Reussner, and B. Vogel-Heuser, “Architecture-based change impact analysis in cross-disciplinary automated production systems,” *Journal of Systems and Software*, vol. 146, pp. 167–185, 2018, doi: 10.1016/j.jss.2018.08.058.

[42] *Researching evolution in industrial plant automation: Scenarios and documentation of the extended pick and place unit*, 2018. [Online]. Available: https://mediatum.ub.tum.de/doc/1468863/document.pdf

[43] D. Garlan, J. M. Barnes, B. Schmerl, and O. Celiku, “Evolution styles: Foundations and tool support for software architecture evolution,” in *2009 Joint Working IEEE/IFIP Conference on Software Architecture & European Conference on Software Architecture*, 2009.

[44] M. Naab, “Enhancing Architecture Design Methods for Improved Flexibility in Long-Living Information Systems,” in 2011, pp. 194–198. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-642-23798-0_19

[45] *Evaluating software architectures: methods and case studies*, 2002.

[46] P. Bengtsson, N. Lassing, J. Bosch, and H. van Vliet, “Architecture-level modifiability analysis (ALMA),” *Journal of Systems and Software*, vol. 69, 1-2, pp. 129–147, 2004, doi: 10.1016/S0164-1212(03)00080-3.

[47] E. Cagno, F. Caron, and M. Mancini, “Risk analysis in plant commissioning: the Multilevel Hazop,” *Reliability Engineering & System Safety*, vol. 77, no. 3, pp. 309–323, 2002, doi: 10.1016/S0951-8320(02)00064-9.

[48] M. Zou, B. Vogel-Heuser, M. Sollfrank, and J. Fischer, Eds., *A Cross-disciplinary Model-Based Systems Engineering Workflow of Automated Production Systems Leveraging Socio-technical Aspects*: IEEE, 2020.

[49] Object Management Group (OMG). [Online]. Available: www.bpmn.org (accessed: 08.2023).

[50] Object Management Group (OMG), *Business Process Model and Notation (BPMN) Version 2.0.* [Online]. Available: http://www.omg.org/spec/BPMN/2.0 (accessed: 08.2023).

[51] M. Chinosi and A. Trombetta, “BPMN: An introduction to the standard,” *Computer Standards & Interfaces*, vol. 34, no. 1, pp. 124–134, 2012, doi: 10.1016/j.csi.2011.06.002.

[52] B. Vogel-Heuser, F. Brodbeck, K. Kugler, J. Passoth, S. Maasen, and J. Reif, “BPMN+I to support decision making in innovation management for automated production systems including technological, multi team and organizational aspects,” *IFAC-PapersOnLine*, vol. 53, no. 2, pp. 10891–10898, 2020, doi: 10.1016/j.ifacol.2020.12.2825.

[53] B. Vogel-Heuser *et al.,* “BPMN++ to support managing organisational, multiteam and systems engineering aspects in cyber physical production systems design and operation,” vol. 8, 2022, doi: 10.1017/dsj.2021.29.

[54] A. Fischer, “Analysis of disciplinary differences in methodical development approaches in Mechatronic Systems,” Bachelor's thesis, Institute of Machine Components and Methods of Development, Graz University of Technology, 2023.

[55] Daniel Angermeier, Christian Bartelt, Otto Bauer, *V-Modell XT - Das deutsche Referenzmodell für Systementwicklungsprojekte.* [Online]. Available: https://www.cio.bund.de/Webs/CIO/DE/digitaler-wandel/Achitekturen_und_Standards/V_modell_xt/v_modell_xt_links_und_downloads/v_modell_xt_links_und_downloads_node.html

[56] B. Vogel-Heuser, *Systems software engineering: [angewandte Methoden des Systementwurfs für Ingenieure]*. München: Oldenbourg, op. 2003.

[57] Z. Yu and V. Rajlich, "Hidden dependencies in program comprehension and change propagation," in *9th International Workshop on Program Comprehension, 2001. IWPC 2001. Proceedings*, Toronto, Ont., Canada, 2001, pp. 293–299.

[58] Institute of Automation and Information Systems, *advacode - Advanced systems engineering for control software as a prerequisite for flexible, adaptive cyberphysical production systems.* [Online]. Available: https://www.mec.ed.tum.de/en/ais/research/current-research-projects/advacode/ (accessed: 10.2023).

[59] Siemens, *TIA Portal Openness: Introduction and Demo Application.* [Online]. Available: https://support.industry.siemens.com/cs/document/108716692/tia-portal-openness-introduction-and-demo-application?dti=0&lc=en-DE (accessed: 10.2023).

[60] *teamtechnik homepage.* [Online]. Available: https://www.teamtechnik.com/ (accessed: 04.2023).

[61] International Organisation for Standardization, *ISO 25010 - Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models.* [Online]. Available: https://www.iso.org/standard/35733.html (accessed: 09.2023).

[62] B. Vogel-Heuser, F. Ocker, and T. Scheuer, "An approach for leveraging Digital Twins in agent-based production systems," *at – Automatisierungstechnik*, vol. 69, no. 12, pp. 1026–1039, 2021, doi: 10.1515/auto-2021-0081.

Appendix A. Abstraction of Industry Partners' Workflow in a Non-standard Decision Tree

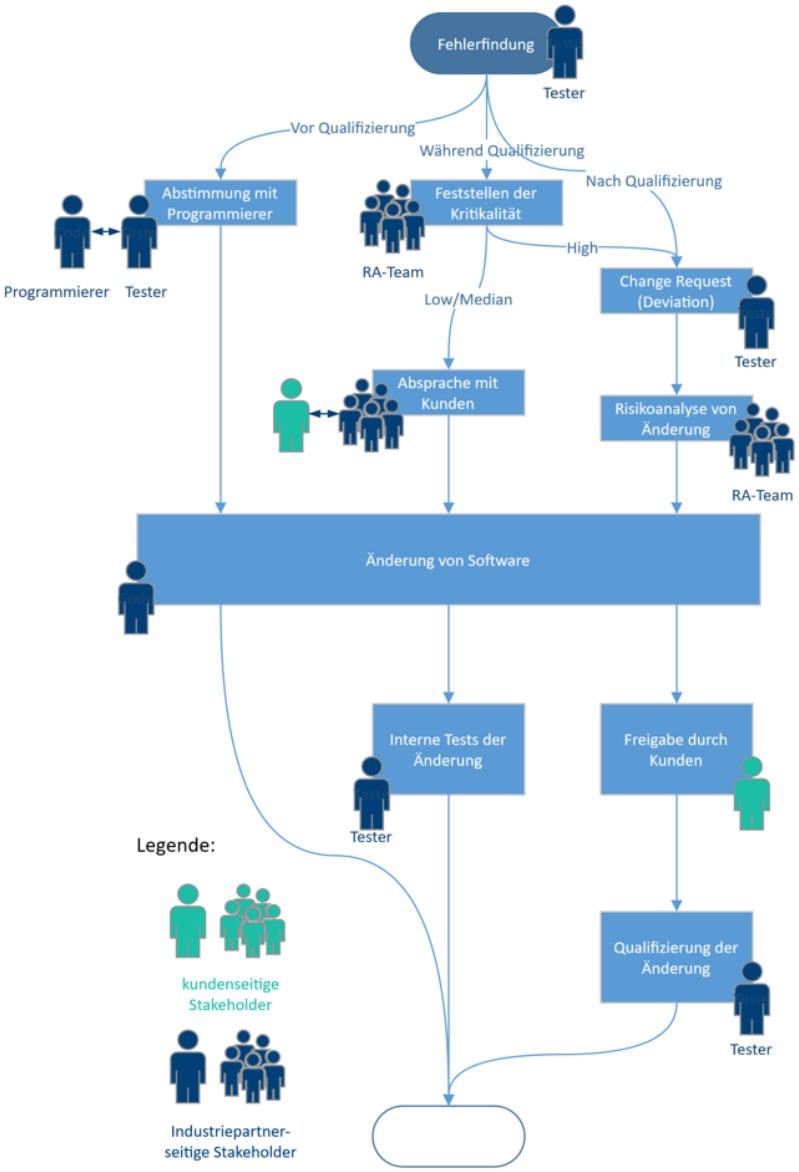

Figure 16: Non-standard Decision Tree model of industry partners' workflow as used for evaluation with industry partners' experts

Appendix B. Abstraction of GAMP-suggested Workflow in a Non-standard Decision Tree

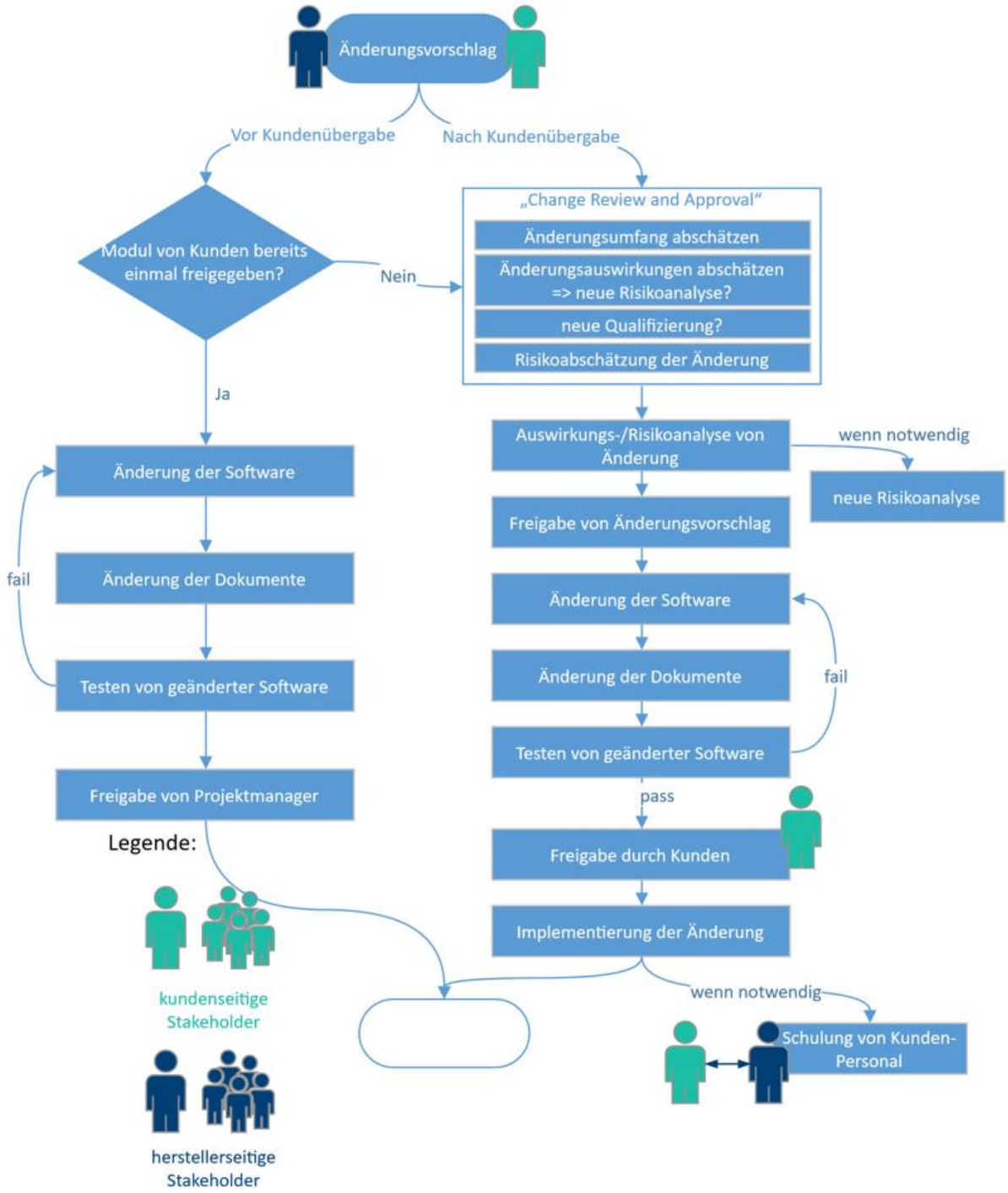

Figure 17: Non-standard Decision Tree model of GAMP-suggested workflow as used for evaluation with industry partners' experts.

Appendix C. Summary of Questionnaire used to Determine Background of Experts in Interviews

1.	Current company and specific domain of expertise through past endeavours?
2.	Mainly contact with in-house projects and/or customer orders?
3.	Experience with changes during development, commissioning and/or operational phase of the aPS life cycle?
4.	Current approach to general software testing during the phases with experience? (In the Context of ‘Who conducts tests?’, ‘under which circumstances are they being performed?’, etc.)
5.	Current approach to required software changes during the phases with experience?
6.	Overall size of company and teams involved at one aPS project?
7.	Experience with customer contact? During which phase of the aPS life cycle?

Table 2: Summary of questionnaire used to determine the specific background of Experts A and B.

Eidesstattliche Erklärung

Ich erkläre, die Arbeit selbständig verfasst und bei der Erstellung dieser Arbeit die einschlägigen Bestimmungen, insbesondere zum Urheberrechtsschutz fremder Beiträge, eingehalten zu haben. Soweit meine Arbeit fremde Beiträge (z. B. Bilder, Zeichnungen, Textpassagen) enthält, erkläre ich, dass diese Beiträge als solche gekennzeichnet sind (z. B. Zitat, Quellenangabe) und ich eventuell erforderlich gewordene Zustimmungen der Urheber zur Nutzung dieser Beiträge in meiner Arbeit eingeholt habe. Die Arbeit wurde nicht bereits in derselben oder einer ähnlichen Fassung an einer anderen Fakultät oder einem anderen Fachbereich im In- und Ausland zur Erlangung eines akademischen Grades eingereicht.

Unterschrift: 

München, den 14.10.2023